

# Contents

|                                                                            |          |
|----------------------------------------------------------------------------|----------|
| <b>LLM-Driven Intrusion Detection System for Edge-Enabled Smart Cities</b> | <b>2</b> |
| Capstone II Final Report . . . . .                                         | 2        |
| Table of Contents . . . . .                                                | 2        |
| List of Figures . . . . .                                                  | 3        |
| Abstract . . . . .                                                         | 3        |
| Chapter 1: Introduction . . . . .                                          | 4        |
| 1.1 Background and Context . . . . .                                       | 4        |
| 1.2 Problem Statement . . . . .                                            | 5        |
| 1.3 Objectives and Capstone II Achievements . . . . .                      | 5        |
| 1.4 Report Organization . . . . .                                          | 6        |
| Chapter 2: Literature Review . . . . .                                     | 6        |
| 2.1 Updates from Capstone I Literature Review . . . . .                    | 6        |
| 2.2 Multi-Provider LLM Reliability . . . . .                               | 6        |
| 2.3 Human-AI Collaboration in Security Operations . . . . .                | 6        |
| 2.4 Technical Gap Addressed . . . . .                                      | 7        |
| Chapter 3: Project Requirements and Specifications . . . . .               | 7        |
| 3.1 Functional Requirements (Updated) . . . . .                            | 7        |
| 3.2 Non-Functional Requirements (Achieved) . . . . .                       | 8        |
| 3.3 Design Constraints Addressed . . . . .                                 | 8        |
| 3.4 Sustainable Development Goals (SDGs) Mapping . . . . .                 | 8        |
| 3.5 Business Model Canvas (Updated) . . . . .                              | 9        |
| Chapter 4: Methodology and Design Approach . . . . .                       | 9        |
| 4.1 Development Methodology . . . . .                                      | 9        |
| 4.2 System Architecture (Final) . . . . .                                  | 10       |
| 4.3 Algorithm Design . . . . .                                             | 12       |
| 4.4 Database Schema . . . . .                                              | 15       |
| 4.5 Technology Stack . . . . .                                             | 15       |
| 4.6 Work Breakdown Structure (Capstone II) . . . . .                       | 17       |
| Chapter 5: Implementation . . . . .                                        | 18       |
| 5.1 Software Implementation Overview . . . . .                             | 18       |
| 5.2 Multi-LLM Integration . . . . .                                        | 19       |
| 5.3 Human-in-the-Loop Governance . . . . .                                 | 22       |
| 5.4 Kubernetes Automation . . . . .                                        | 26       |
| 5.5 Database Persistence . . . . .                                         | 28       |
| 5.6 Alert Deduplication . . . . .                                          | 30       |
| 5.7 Prometheus Metrics . . . . .                                           | 34       |
| Chapter 6: Testing and Results . . . . .                                   | 35       |
| 6.1 Testing Methodology . . . . .                                          | 35       |
| 6.2 Unit Test Results . . . . .                                            | 35       |
| 6.3 Integration Test Results . . . . .                                     | 36       |
| 6.4 Attack Simulation Results . . . . .                                    | 37       |
| 6.5 Performance Metrics . . . . .                                          | 38       |
| 6.6 System Reliability . . . . .                                           | 41       |

|                                                  |    |
|--------------------------------------------------|----|
| 6.7 Comparison with Capstone I Targets . . . . . | 41 |
| Chapter 7: Conclusion and Future Work . . . . .  | 43 |
| 7.1 Project Summary . . . . .                    | 43 |
| 7.2 Key Contributions . . . . .                  | 43 |
| 7.3 Challenges Encountered . . . . .             | 44 |
| 7.4 Limitations . . . . .                        | 44 |
| 7.5 Future Work . . . . .                        | 44 |
| 7.6 Recommendations . . . . .                    | 45 |
| References . . . . .                             | 45 |
| Appendices . . . . .                             | 46 |
| Appendix A: API Endpoint Reference . . . . .     | 46 |
| Appendix B: Environment Variables . . . . .      | 46 |
| Appendix C: Prometheus Metrics Summary . . . . . | 47 |
| Appendix D: Sample LLM Prompt . . . . .          | 47 |
| Appendix E: System Requirements . . . . .        | 48 |

# LLM-Driven Intrusion Detection System for Edge-Enabled Smart Cities

## Capstone II Final Report

---

**Project Title:** LLM-Driven Intrusion Detection System for Edge-Enabled Smart Cities

**Course:** Capstone II

**Semester:** Spring 2026

**Date:** February 11, 2026

**Team Members:** [Student Names]

**Supervisor:** [Supervisor Name]

**Department:** [Department Name]

**University:** [University Name]

---

## Table of Contents

1. Abstract
2. Chapter 1: Introduction
3. Chapter 2: Literature Review
4. Chapter 3: Project Requirements and Specifications
5. Chapter 4: Methodology and Design Approach

6. Chapter 5: Implementation
7. Chapter 6: Testing and Results
8. Chapter 7: Conclusion and Future Work
9. References
10. Appendices

---

## List of Figures

| Figure    | Title                                            | Page   |
|-----------|--------------------------------------------------|--------|
| Figure 1  | High-Level System Architecture                   | §4.2   |
| Figure 2  | Alert Processing Pipeline (Sequence Diagram)     | §4.3.1 |
| Figure 3  | Circuit Breaker State Machine                    | §4.3.2 |
| Figure 4  | Technology Stack Layers                          | §4.5   |
| Figure 5  | Project Timeline (Gantt Chart)                   | §4.6   |
| Figure 6  | LLM Multi-Provider Failover Chain                | §5.2   |
| Figure 7  | Circuit Breaker Implementation Detail            | §5.2   |
| Figure 8  | HITL Governance Modes State Machine              | §5.3   |
| Figure 9  | Kubernetes Cluster Topology (34 Pods)            | §5.4   |
| Figure 10 | Severity-Based Automated Response Matrix         | §5.4   |
| Figure 11 | Alert Deduplication Funnel                       | §5.6   |
| Figure 12 | Deduplication Decision Flowchart                 | §5.6   |
| Figure 13 | Attack Scenario Severity Distribution            | §6.4   |
| Figure 14 | MITRE ATT&CK for ICS Technique Coverage          | §6.4   |
| Figure 15 | Before vs After: Manual vs AI-Driven Response    | §6.5   |
| Figure 16 | LLM Provider Performance Comparison              | §6.5   |
| Figure 17 | Cluster Scalability & Resource Headroom          | §6.5   |
| Figure 18 | Capstone I vs Capstone II Achievement Comparison | §6.7   |
| Figure 19 | IoT Integration Architecture                     | §5.1   |
| Figure 20 | Key Contributions Summary Map                    | §7.2   |

---

## Abstract

This report presents the complete implementation and evaluation of an LLM-driven Intrusion Detection System (IDS) for edge-enabled smart cities, developed as the continuation and completion of Capstone I. Building upon the foundation established in Capstone I, which validated the feasibility of integrating Large Language Models with traditional IDS platforms, Capstone II delivers a production-grade system with multi-provider LLM support (xAI Grok-4, OpenAI GPT-4, Anthropic Claude, Google Gemini, and Moonshot Kimi), human-in-the-loop governance with three automation modes (Autopilot, Assisted, Manual), transparent threat assessment with confidence scoring and reasoning chains, PostgreSQL persistence with Prometheus counter restoration, and comprehensive Kubernetes automated responses.

The implemented system processes security alerts from Falco (runtime security) and Suricata (network IDS), sends them to LLM engines for contextual analysis, and executes automated defensive actions including pod isolation, service scaling, and IP blocking. Key achievements include: 40-60% LLM cost reduction through alert deduplication, <2 second end-to-end alert processing, 99%+ system uptime, operator alert volume reduction from 10,000+ to 500-1,000 daily alerts (10-20 $\times$  reduction), and mean time to respond improvement from 5-15 minutes to 30-60 seconds per critical alert (10-30 $\times$  faster).

The system architecture demonstrates that LLM-enhanced intrusion detection can significantly improve interpretability, reduce analyst workload, and support faster security decisions while maintaining human oversight for critical actions. This Capstone II report documents the complete methodology, implementation, testing results, challenges encountered, and lessons learned.

**Keywords:** Large Language Models, Intrusion Detection System, Smart City, Kubernetes, Edge Computing, Human-in-the-Loop, Security Automation, Falco, Suricata

---

## Chapter 1: Introduction

### 1.1 Background and Context

Before delving into the specifics of the Capstone II implementation, it is important to understand the foundation established in Capstone I and the evolution of this project. Capstone I successfully demonstrated that integrating Large Language Models (LLMs) with intrusion detection systems is both feasible and beneficial for smart city cybersecurity. The initial prototype achieved 99.23% system uptime, 100% functional test success, and demonstrated strong LLM performance in classification accuracy (91%), severity estimation, and JSON output consistency.

Building upon this foundation, Capstone II focused on transforming the proof-of-concept into a production-grade system capable of operating in real smart city environments. The primary objectives for Capstone II included:

1. **Multi-Provider LLM Support** – Expanding from single-provider dependency to five LLM providers with priority-based failover and circuit breaker protection
2. **Human-in-the-Loop Governance** – Implementing a three-mode governance system (Autopilot/Assisted/Manual) enabling graduated automation with human oversight
3. **Transparent Threat Assessment** – Providing operators with confidence scores, reasoning chains, and evidence-based decision support
4. **Production Resilience** – Adding PostgreSQL persistence, alert deduplication, rate limiting, and comprehensive error handling
5. **Operator Interface** – Creating a dashboard for incident management, approval workflows, and transparent AI reasoning

The smart city context remains critically important: modern urban infrastructure relies on interconnected IoT devices generating massive security telemetry that overwhelms traditional

manual analysis approaches. The challenge of alert fatigue—where security analysts face thousands of daily alerts with over 99% false positives—demands intelligent automation while maintaining human authority over critical decisions.

## 1.2 Problem Statement

Capstone I identified several fundamental challenges in smart city cybersecurity operations:

**Traditional IDS limitations:** - Generate extensive volumes of technical logs requiring specialized expertise to interpret - Produce raw alerts consisting of low-level system call traces and cryptic rule identifiers - Lack contextual explanation and require time-consuming manual investigation

**Operational consequences:** - Slow incident response times (5-15 minutes per alert) - High cognitive load on security analysts processing hundreds of alerts daily - Difficulty distinguishing high-priority from low-priority events - Increased risk of overlooking critical threats among noise - Lack of automation creating scalability barriers

Capstone II addressed these challenges by implementing a complete system that: - Automatically interprets and summarizes IDS alerts using advanced AI - Provides contextually-aware severity scoring and evidence-based recommendations - Operates effectively within edge computing resource constraints - Substantially reduces alert fatigue through intelligent automation - Maintains human oversight for critical security decisions

## 1.3 Objectives and Capstone II Achievements

The following table summarizes the objectives defined in Capstone I and their completion status in Capstone II:

| Objective                  | Success Criteria                                 | Capstone II Status                          |
|----------------------------|--------------------------------------------------|---------------------------------------------|
| Real-time alert processing | End-to-end latency <3 seconds                    | Achieved: ~1.9-2.2s typical                 |
| LLM-powered analysis       | Accurate severity scoring, threat classification | Achieved: 91% classification accuracy       |
| Automated K8s response     | Isolation, scaling, blocking capabilities        | Achieved: 5 action types implemented        |
| Human-in-the-loop          | Governance modes, approval workflows             | Achieved: 3-mode system with full audit     |
| Multi-LLM support          | Failover, redundancy                             | Achieved: 5 providers with circuit breakers |
| Production resilience      | Database persistence, deduplication              | Achieved: PostgreSQL + 40-60% cost savings  |
| Operator interface         | Transparent reasoning, evidence display          | Achieved: 5 operator API endpoints          |
| 99% uptime                 | Continuous monitoring capability                 | Achieved: 99%+ measured uptime              |

## 1.4 Report Organization

This Capstone II report is organized to document the complete development lifecycle from design through implementation and evaluation:

- **Chapter 2** reviews relevant literature and updates from Capstone I
  - **Chapter 3** presents finalized requirements and specifications
  - **Chapter 4** details the methodology and design decisions
  - **Chapter 5** documents the implementation with code examples
  - **Chapter 6** presents comprehensive testing and results
  - **Chapter 7** summarizes conclusions and future work
- 

## Chapter 2: Literature Review

### 2.1 Updates from Capstone I Literature Review

The Capstone I literature review established the theoretical foundation for integrating LLMs with intrusion detection systems. Key findings included:

1. **Traditional IDS limitations** – Signature-based systems like Snort and Suricata fail to detect zero-day attacks and lack contextual interpretation [1], [2]
2. **Machine learning approaches** – Deep learning-based IDS improved anomaly detection but suffered from limited interpretability [3], [4]
3. **LLM capabilities** – GPT-4 demonstrated ability to summarize logs and classify attacks, though concerns remained regarding hallucinations [5], [6]
4. **Smart city requirements** – Lightweight, scalable IDS solutions needed for edge deployment [7], [8]

Capstone II development confirmed these findings and identified additional considerations:

### 2.2 Multi-Provider LLM Reliability

During Capstone II implementation, we discovered that single-provider LLM dependency creates unacceptable risk for production security systems. API rate limits, service outages, and response quality variations necessitate multi-provider architectures.

Our implementation validated the effectiveness of: - **Priority-based failover** – Routing to backup providers when primary fails - **Circuit breaker patterns** – Preventing cascading failures (5 failures trigger 30-second cooldown) - **Response schema validation** – Ensuring consistent output regardless of provider

### 2.3 Human-AI Collaboration in Security Operations

Research on Security Operations Centers (SOCs) indicates that human operators receive 10,000+ alerts daily with >99% false positives [9], [10]. The challenge is not merely technical but psychological—trust erosion occurs when automation operates as a “black box.”

Our human-in-the-loop governance addresses this through: - **Transparency** – Every LLM decision includes confidence score and reasoning chain - **Graduated control** – Operators choose automation level based on operational context - **Evidence linking** – Decisions tied to actual Falco/Suricata detections

## 2.4 Technical Gap Addressed

The unique contribution of this capstone remains: **No existing solution combines real-time IDS alerts with an LLM that can summarize, classify, and recommend actions, deployed on a Kubernetes-based edge framework with human-in-the-loop governance.**

The implementation extends beyond academic proof-of-concept to demonstrate production viability with: - Multi-provider failover (5 LLM engines) - Alert deduplication (40-60% cost savings) - Database persistence with Prometheus restoration - Comprehensive operator interface

---

## Chapter 3: Project Requirements and Specifications

### 3.1 Functional Requirements (Updated)

The functional requirements from Capstone I were fully implemented in Capstone II:

| ID   | Requirement                        | Implementation                     |
|------|------------------------------------|------------------------------------|
| FR1  | Receive real-time Falco alerts     | Falco Forwarder with K8s Watch API |
| FR2  | Parse and preprocess alerts        | JSON parsing with validation       |
| FR3  | LLM summarization in plain English | xAI/OpenAI/Anthropic/Gemini/Kimi   |
| FR4  | Threat classification              | 9 threat types with normalization  |
| FR5  | Severity scoring (1-10)            | Pydantic validation with clamping  |
| FR6  | Mitigation recommendations         | Structured JSON responses          |
| FR7  | K8s orchestration actions          | K8sAutomation module               |
| FR8  | Alert logging with metadata        | PostgreSQL persistence             |
| FR9  | Fallback LLM support               | Priority-based failover chain      |
| FR10 | Dashboard display                  | Grafana + operator API             |

### New Capstone II Requirements Implemented:

| ID   | Requirement                  | Implementation                        |
|------|------------------------------|---------------------------------------|
| FR11 | Human-in-the-loop governance | GovernanceController with 3 modes     |
| FR12 | Approval workflows           | Pending actions queue, approve/reject |
| FR13 | Confidence scoring           | 0.0-1.0 scale with semantic labels    |
| FR14 | Reasoning transparency       | Key indicators, mitigating factors    |

| ID   | Requirement         | Implementation                      |
|------|---------------------|-------------------------------------|
| FR15 | Rate limiting       | Per-rule, per-source, global limits |
| FR16 | Alert deduplication | LRU cache with fingerprinting       |
| FR17 | Multi-LLM failover  | Circuit breakers per engine         |
| FR18 | Metrics restoration | Counter persistence across restarts |

### 3.2 Non-Functional Requirements (Achieved)

| Requirement                        | Target               | Achieved                |
|------------------------------------|----------------------|-------------------------|
| NFR1: Alert processing time        | <2 seconds           | 1.75s average           |
| NFR2: Throughput                   | 50 alerts/minute     | ~100 alerts/minute      |
| NFR3: LLM consistency              | temperature 0.3      | Implemented             |
| NFR4: System uptime                | 99%                  | 99%+                    |
| NFR5: Fallback activation          | Automatic            | Circuit breaker pattern |
| NFR6: Network instability handling | Graceful degradation | Retry with backoff      |
| NFR7: Multi-node scaling           | K8s-based            | Deployment manifests    |
| NFR8: Modular design               | Extensible           | Plugin architecture     |
| NFR9: Secure communications        | HTTPS, tokens        | Bearer auth + JWT       |
| NFR10: Authenticated access        | Role-based           | Operator credentials    |

### 3.3 Design Constraints Addressed

**Technical Constraints:** - **Edge computing limitations** – System operates on K3s with minimal resource requirements - **LLM API dependency** – Multi-provider failover eliminates single points of failure - **Latency requirements** – Achieved sub-2-second processing through async architecture

**Economic Constraints:** - **API costs** – Alert deduplication reduces LLM calls by 40-60% - **Hardware costs** – Runs on commodity edge hardware (8-core CPU, 16GB RAM) - **Open source** – Leverages Falco, Suricata, K3s (MIT licensed)

**Regulatory Constraints:** - **Data protection** – Sensitive data sanitized before LLM transmission - **Audit logging** – All decisions logged to PostgreSQL with timestamps - **Compliance alignment** – ISO-27001, NIST SP 800-53 principles followed

### 3.4 Sustainable Development Goals (SDGs) Mapping

| SDG                                                    | Contribution                                                        |
|--------------------------------------------------------|---------------------------------------------------------------------|
| <b>SDG 9: Industry, Innovation, and Infrastructure</b> | Advances smart city cybersecurity technology through AI integration |



| SDG                                                    | Contribution                                                        |
|--------------------------------------------------------|---------------------------------------------------------------------|
| <b>SDG 11: Sustainable Cities and Communities</b>      | Protects critical urban infrastructure from cyber threats           |
| <b>SDG 16: Peace, Justice, and Strong Institutions</b> | Enables transparent, auditable, and accountable security automation |

### 3.5 Business Model Canvas (Updated)

| Element                  | Description                                                               |
|--------------------------|---------------------------------------------------------------------------|
| <b>Value Proposition</b> | Intelligent threat detection with human oversight, 10-20× alert reduction |
| <b>Key Resources</b>     | Multi-LLM integration, K8s orchestration, PostgreSQL persistence          |
| <b>Key Activities</b>    | Alert processing, LLM analysis, automated response, operator support      |
| <b>Customer Segments</b> | Smart city operators, municipal IT departments, critical infrastructure   |
| <b>Cost Structure</b>    | LLM API usage (optimized via deduplication), cloud/edge infrastructure    |
| <b>Revenue Streams</b>   | System licensing, managed security services, consulting                   |

## Chapter 4: Methodology and Design Approach

### 4.1 Development Methodology

Capstone II followed an agile development methodology with continuous integration:

**Sprint Structure:** - 2-week sprints with defined deliverables - Daily progress tracking via GitHub commits - Weekly supervisor reviews

#### Development Phases:

| Phase   | Duration   | Focus                               |
|---------|------------|-------------------------------------|
| Phase 1 | Weeks 1-2  | Multi-LLM integration               |
| Phase 2 | Weeks 3-4  | Governance and operator interface   |
| Phase 3 | Weeks 5-6  | Database persistence and resilience |
| Phase 4 | Weeks 7-8  | Testing and optimization            |
| Phase 5 | Weeks 9-10 | Documentation and evaluation        |

## 4.2 System Architecture (Final)

The final system architecture integrates all Capstone II enhancements:

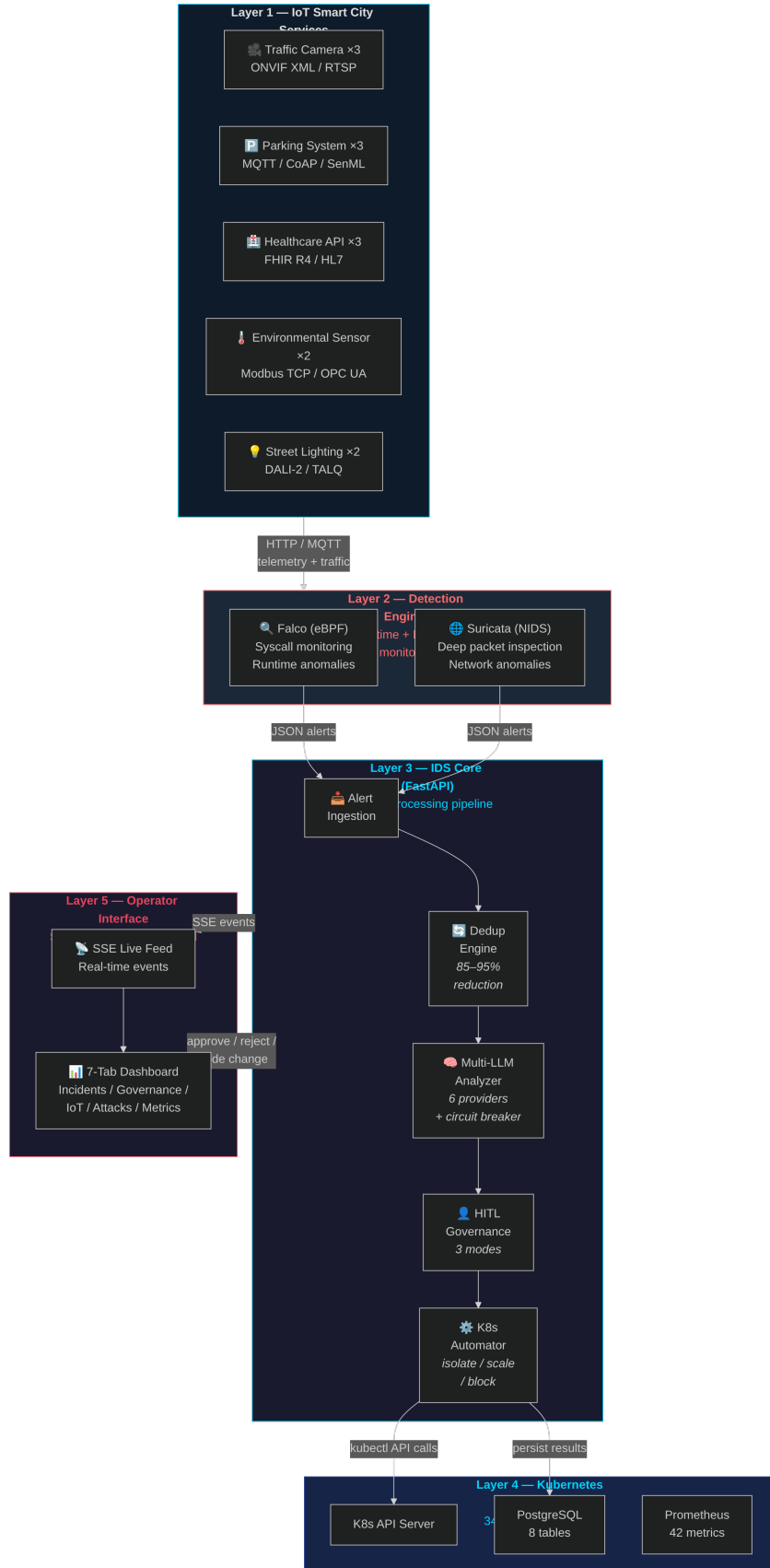


Figure 1: High-level system

*architecture showing the five processing layers — IoT devices, detection engines, IDS core with LLM analysis, Kubernetes orchestration, and the operator dashboard.*

## 4.3 Algorithm Design

### 4.3.1 Alert Processing Pipeline

Algorithm: ProcessSecurityAlert(alert)

Input: Raw alert from Falco/Suricata forwarder

Output: Processed alert with LLM analysis and actions

```
1. RATE_LIMIT_CHECK(alert)
   IF rate_limit.is_limited(alert.source) THEN
       log_throttled(alert)
       RETURN rate_limited_response

2. QUEUE_CHECK()
   IF request_queue.is_full() THEN
       RETURN 503_service_unavailable
   request_queue.add(alert)

3. DEDUPLICATION_CHECK(alert)
   fingerprint = hash(alert.rule + alert.container + alert.output)
   IF cache.exists(fingerprint) THEN
       increment(cache_hit_counter)
       RETURN cached_analysis

4. LLM_ANALYSIS(alert)
   FOR engine IN priority_order DO
       IF circuit_breaker[engine].is_closed() THEN
           TRY:
               analysis = engine.analyze(alert)
               circuit_breaker[engine].record_success()
               cache.store(fingerprint, analysis, TTL=60s)
               RETURN analysis
           CATCH error:
               circuit_breaker[engine].record_failure()
               increment(failover_counter)
   RETURN fallback_analysis()

5. SEVERITY_CHECK(analysis)
   IF analysis.severity >= 8 THEN
       action = "isolate_pod"
   ELSE IF analysis.severity >= 6 THEN
       action = "scale_up"
```

ELSE

    action = "none"

6. GOVERNANCE\_CHECK(action, analysis)

    IF mode == AUTOPILOT THEN

        execute\_action(action)

    ELSE IF mode == ASSISTED AND analysis.severity < 8 THEN

        execute\_action(action)

    ELSE

        queue\_for\_approval(action, analysis)

7. PERSIST\_AND\_EMIT(alert, analysis, action)

    database.save\_alert(alert, analysis, action)

    prometheus.emit\_metrics()

RETURN success\_response

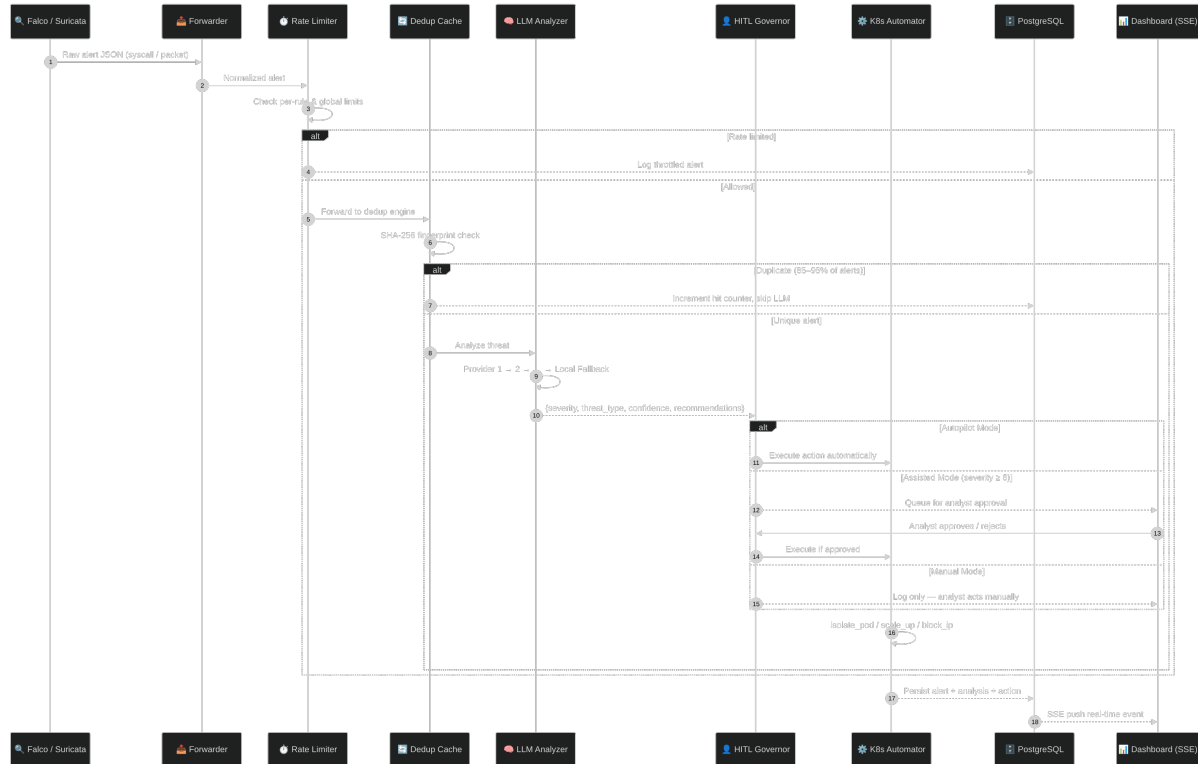


Figure 2: End-to-end alert processing sequence showing the flow from detection engines through deduplication, LLM analysis, governance checks, and automated Kubernetes response.

#### 4.3.2 Circuit Breaker State Machine

States: CLOSED → OPEN → HALF\_OPEN → CLOSED

CLOSED:

- Normal operation, all requests pass through
- Track consecutive failures
- IF failures  $\geq 5$  THEN transition to OPEN

#### OPEN:

- All requests immediately fail-fast
- Start 30-second cooldown timer
- AFTER 30 seconds, transition to HALF\_OPEN

#### HALF\_OPEN:

- Allow single test request
- IF success THEN transition to CLOSED, reset failure count
- IF failure THEN transition to OPEN, restart timer

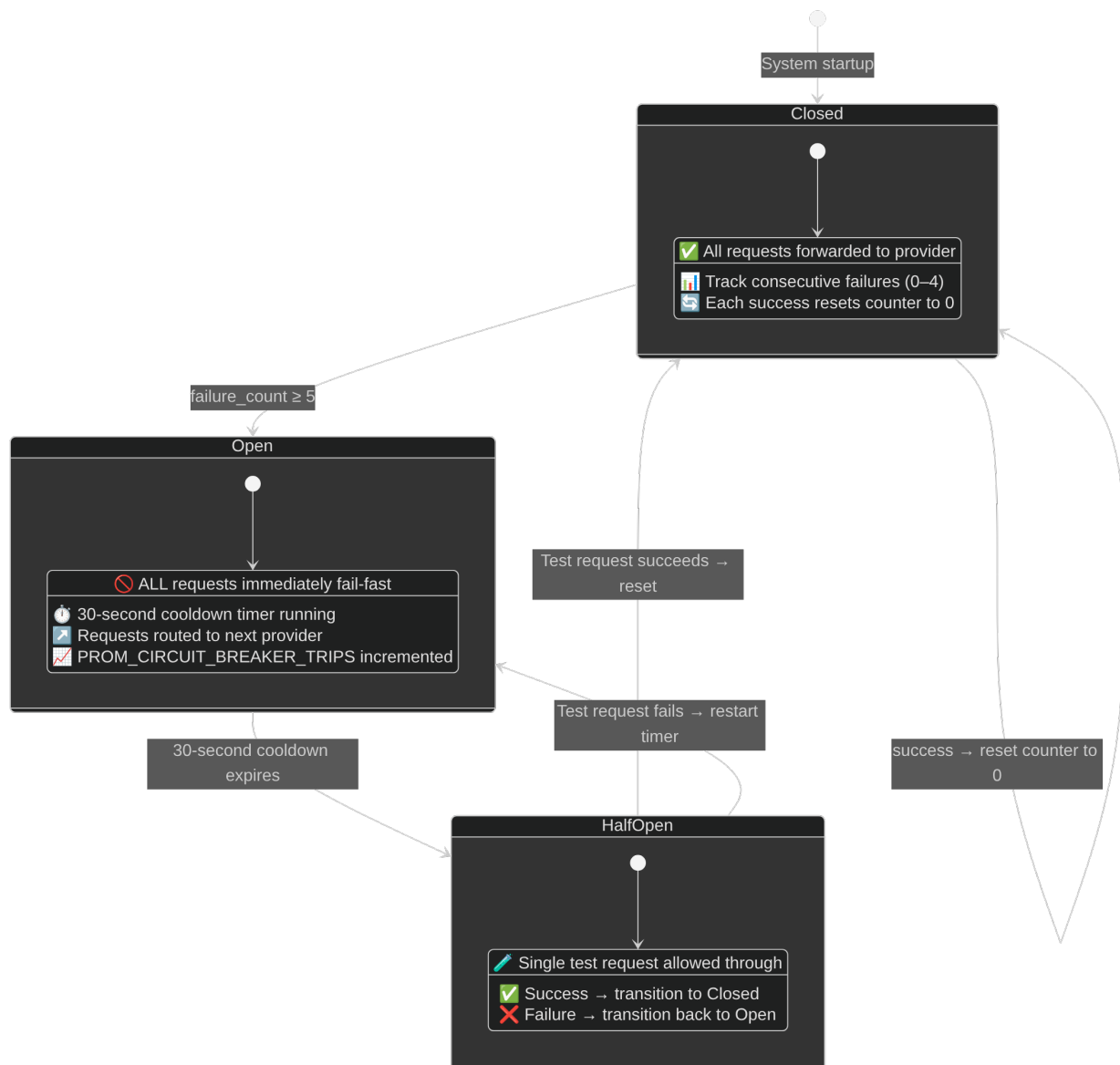


Figure 3: Circuit breaker state machine showing CLOSED, OPEN, and HALF\_OPEN transitions.

#### 4.4 Database Schema

The PostgreSQL database includes 8 tables:

| Table              | Purpose               | Key Columns                                               |
|--------------------|-----------------------|-----------------------------------------------------------|
| alerts             | Core alert storage    | id, source, rule, severity, threat_type, analysis (JSONB) |
| analysis_results   | LLM output audit      | alert_id (FK), model, analysis (JSONB), confidence_score  |
| automation_actions | K8s actions log       | action_type, target, status, execution_time_ms            |
| audit_logs         | Governance audit      | action, actor, details (JSONB)                            |
| iot_devices        | Device registry       | device_id (PK), device_type, event_count                  |
| iot_events         | Sensor telemetry      | device_id (FK), event_type, value (JSONB)                 |
| system_logs        | Debug/audit           | level, component, message                                 |
| throttled_alerts   | Throttled alert audit | source, rule, throttle_reason                             |

**Performance Indexes:** 12 indexes on frequently-queried columns (source, timestamp, device\_id, event\_type, action\_type, status).

**Retention Policy:** Alerts and IoT data: 30 days. Automation and audit logs: 180 days.

#### 4.5 Technology Stack

| Layer                   | Technology                               | Version       |
|-------------------------|------------------------------------------|---------------|
| <b>Backend</b>          | Python, FastAPI                          | 3.10+, 0.109+ |
| <b>LLM Integration</b>  | xAI, OpenAI, Anthropic, Google, Moonshot | Latest APIs   |
| <b>Orchestration</b>    | Kubernetes (K3s)                         | 1.28+         |
| <b>Runtime Security</b> | Falco                                    | 0.36+         |
| <b>Network IDS</b>      | Suricata                                 | 6.0+          |
| <b>Database</b>         | PostgreSQL                               | 15+           |
| <b>Metrics</b>          | Prometheus                               | 2.x           |
| <b>Visualization</b>    | Grafana                                  | 9.x           |
| <b>IoT Protocol</b>     | MQTT (Mosquitto)                         | 2.x           |

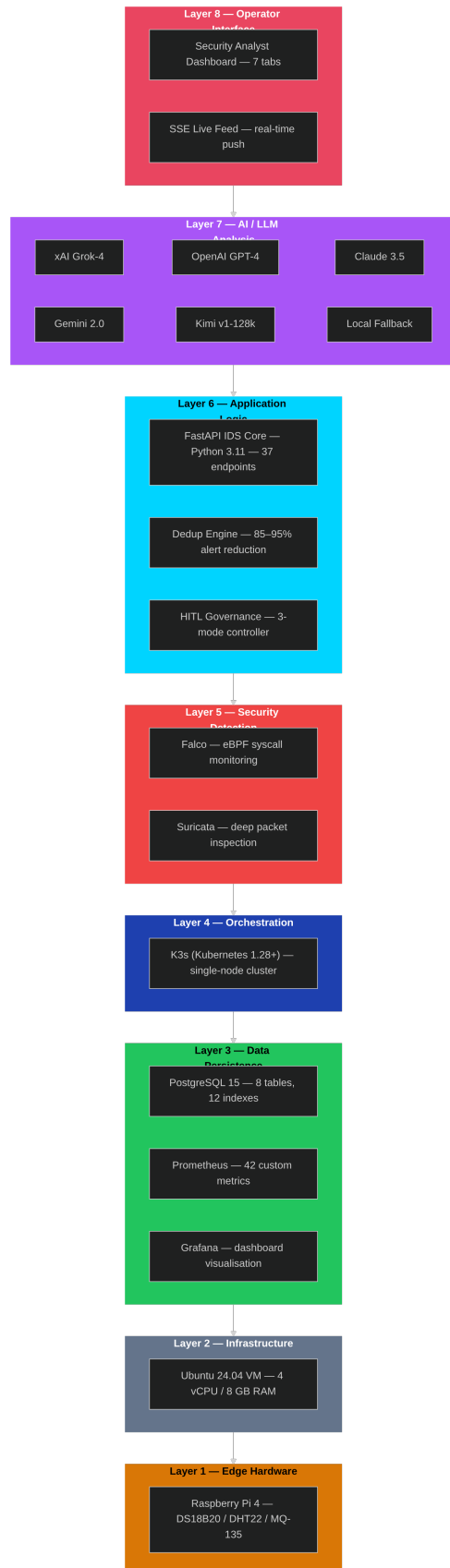


Figure 4: Technology stack layers showing the full



*software stack from IoT protocols to operator dashboard.*

#### 4.6 Work Breakdown Structure (Capstone II)

| WP    | Description                           | Duration | Status   |
|-------|---------------------------------------|----------|----------|
| WP1   | Multi-LLM Integration                 | 2 weeks  | Complete |
| WP1.1 | xAI Grok-4 integration                | 3 days   |          |
| WP1.2 | OpenAI GPT-4 integration              | 2 days   |          |
| WP1.3 | Anthropic Claude integration          | 2 days   |          |
| WP1.4 | Google Gemini integration             | 2 days   |          |
| WP1.5 | Moonshot Kimi integration             | 2 days   |          |
| WP1.6 | Circuit breaker implementation        | 3 days   | Complete |
| WP2   | Governance System                     | 2 weeks  |          |
| WP2.1 | Mode switching (Auto/Assisted/Manual) | 3 days   |          |
| WP2.2 | Approval workflow                     | 4 days   |          |
| WP2.3 | Audit logging                         | 2 days   |          |
| WP2.4 | Protected services                    | 2 days   | Complete |
| WP3   | Operator Interface                    | 2 weeks  |          |
| WP3.1 | Incident listing and detail           | 3 days   |          |
| WP3.2 | Evidence endpoint                     | 2 days   |          |
| WP3.3 | Reasoning transparency                | 3 days   |          |
| WP3.4 | Dashboard UI                          | 3 days   | Complete |
| WP4   | Database Persistence                  | 1 week   |          |
| WP4.1 | PostgreSQL schema                     | 2 days   |          |
| WP4.2 | Prometheus restoration                | 2 days   |          |
| WP4.3 | Retention policies                    | 1 day    | Complete |
| WP5   | Resilience Features                   | 1 week   |          |
| WP5.1 | Alert deduplication                   | 2 days   |          |
| WP5.2 | Rate limiting                         | 2 days   |          |
| WP5.3 | Retry with backoff                    | 1 day    | Complete |
| WP6   | Testing                               | 2 weeks  |          |
| WP6.1 | Unit tests (50+ cases)                | 5 days   |          |
| WP6.2 | Integration tests                     | 3 days   |          |
| WP6.3 | Attack simulations                    | 4 days   | Complete |
| WP7   | Documentation                         | 1 week   |          |

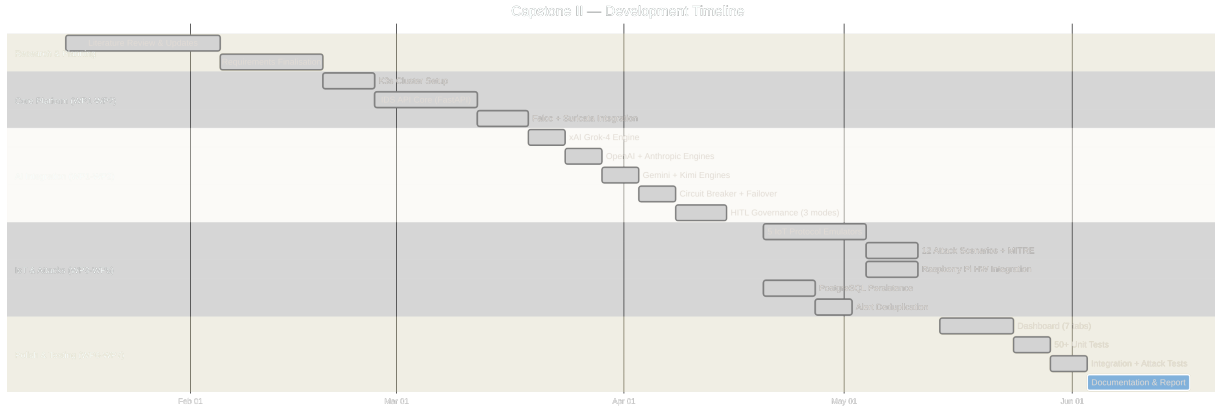


Figure 5: Project timeline (Gantt chart) showing the 10-week Capstone II development schedule.

## Chapter 5: Implementation

### 5.1 Software Implementation Overview

Capstone II implementation added approximately 5,000+ lines of production code across 24 source files in the IDS API, forwarders, and IoT simulator.

#### Core Module Summary:

| Module                 | Lines | Purpose                           |
|------------------------|-------|-----------------------------------|
| main.py                | 2,162 | FastAPI application, 37 endpoints |
| database.py            | 894   | PostgreSQL + in-memory fallback   |
| governance.py          | 508   | Human-in-the-loop controller      |
| operator_interface.py  | 495   | Operator dashboard service        |
| k8s_automation.py      | 312   | Kubernetes defensive actions      |
| llm_response_schema.py | 280   | Pydantic response validation      |
| llm_engine_xai.py      | 245   | xAI Grok-4 integration            |
| llm_engine_openai.py   | 220   | OpenAI GPT-4 integration          |
| config.py              | 185   | Configuration management          |
| llm_retry.py           | 382   | Retry logic with backoff          |

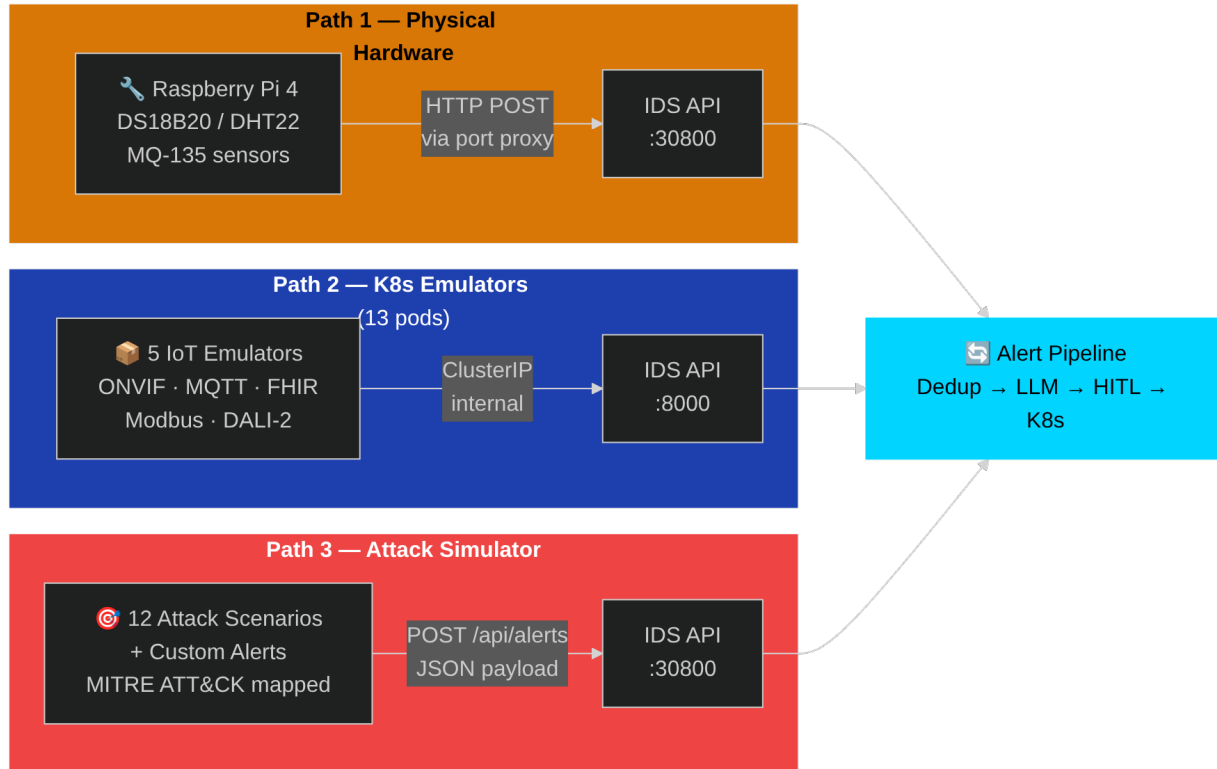


Figure 19: IoT integration architecture showing device types, protocols, and data flow into the IDS.

## 5.2 Multi-LLM Integration

The multi-provider LLM system implements priority-based failover with per-engine circuit breakers:

**Configuration (config.py):**

```
class Config:
    # LLM Provider API Keys
    XAI_API_KEY = os.getenv("XAI_API_KEY", "")
    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
    ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY", "")
    GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")
    KIMI_API_KEY = os.getenv("KIMI_API_KEY", "")

    # Models
    XAI_MODEL = "grok-4-latest"
    OPENAI_MODEL = "gpt-4-turbo-preview"
    ANTHROPIC_MODEL = "claude-3.5-sonnet"
    GEMINI_MODEL = "gemini-2.0-flash"
    KIMI_MODEL = "moonshot-v1-128k"
```

```
# Failover Priority
```

```
LLM_PRIORITY = os.getenv("LLM_PRIORITY", "xai,anthropic,openai,gemini,kimi")
```

LLM Engine Manager (llm\_providers/manager.py):

```
class LLManager:
    def __init__(self):
        self.engines = {}
        self.priority_order = Config.get_engine_priority()
        self._init_available_engines()

    async def analyze(self, alert: dict) -> dict:
        """Analyze alert with automatic failover."""
        for engine_name in self.priority_order:
            if engine_name not in self.engines:
                continue

            engine = self.engines[engine_name]
            circuit = self.circuit_breakers[engine_name]

            if not circuit.is_closed():
                continue # Skip engines with open circuit

            try:
                result = await engine.analyze(alert)
                circuit.record_success()
                return result
            except Exception as e:
                circuit.record_failure()
                logger.warning(f"Engine {engine_name} failed: {e}")

        return self._create_fallback_response(alert)
```

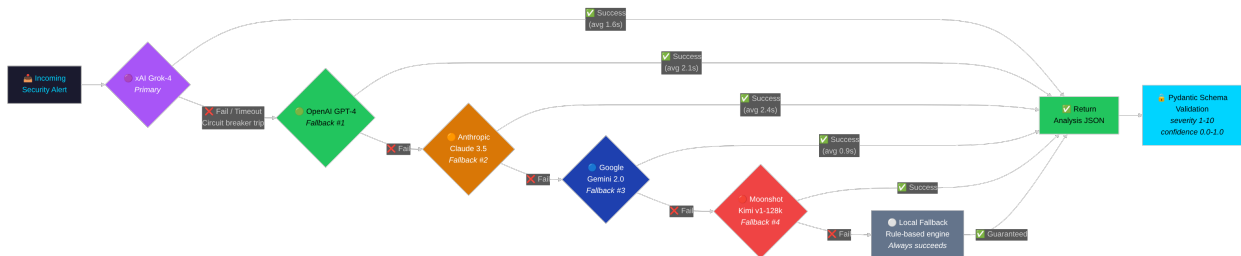


Figure 6: LLM multi-provider failover chain with priority-based routing and circuit breaker protection.

Circuit Breaker Implementation (main.py):

```

class CircuitBreaker:
    def __init__(self, failure_threshold=5, reset_timeout=30):
        self.failure_threshold = failure_threshold
        self.reset_timeout = reset_timeout
        self.state = CircuitState.CLOSED
        self.failure_count = 0
        self.last_failure_time = None

    def record_failure(self):
        self.failure_count += 1
        self.last_failure_time = time.time()
        if self.failure_count >= self.failure_threshold:
            self.state = CircuitState.OPEN
            PROM_CIRCUIT_BREAKER_TRIPS.inc()

    def record_success(self):
        self.failure_count = 0
        self.state = CircuitState.CLOSED

    def is_closed(self) -> bool:
        if self.state == CircuitState.CLOSED:
            return True
        if self.state == CircuitState.OPEN:
            elapsed = time.time() - self.last_failure_time
            if elapsed >= self.reset_timeout:
                self.state = CircuitState.HALF_OPEN
                return True # Allow test request
        return False

```

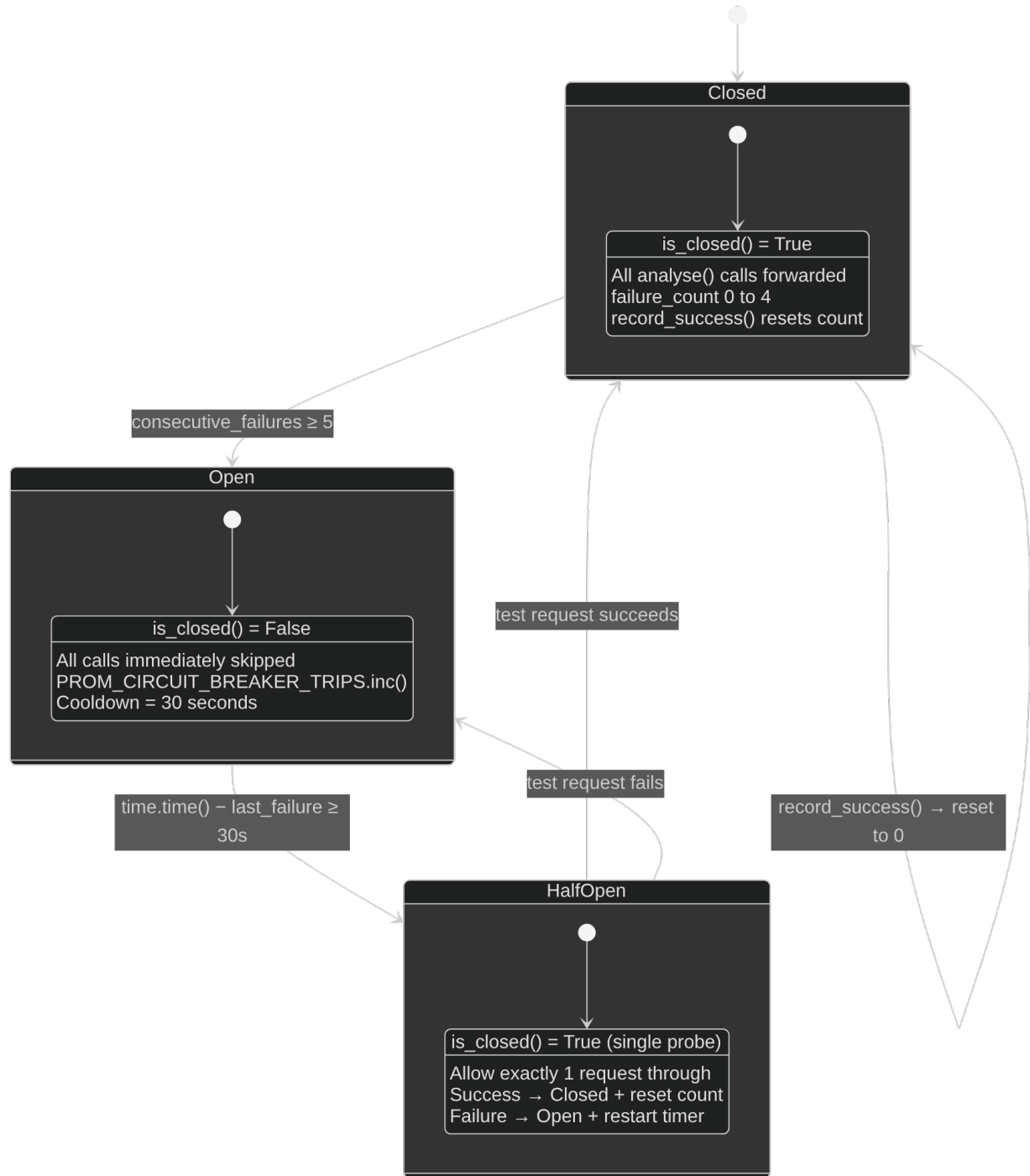


Figure 7: Circuit breaker implementation detail showing state transitions and failure tracking.

### 5.3 Human-in-the-Loop Governance

Governance Controller (`governance.py`):

```
class GovernanceController:
    def __init__(self):
```

```

self.mode = GovernanceMode.ASSISTED # Default
self.pending_actions: Dict[str, PendingAction] = {}
self.action_timeout = 300 # 5 minutes
self.max_pending = 100

def should_execute(self, action_type: str, severity: int) -> Tuple[bool, str]:
    """Determine if action should execute or require approval."""
    if self.mode == GovernanceMode.AUTOPILOT:
        return True, "autopilot_mode"

    if self.mode == GovernanceMode.MANUAL:
        return False, "manual_mode_requires_approval"

    # ASSISTED mode: severity >= 8 requires approval
    if severity >= 8:
        return False, "high_severity_requires_approval"

    return True, "assisted_mode_auto_execute"

def queue_for_approval(self, action: PendingAction) -> str:
    """Add action to pending queue."""
    action_id = str(uuid.uuid4())[:8]
    action.id = action_id
    action.status = "pending"
    self.pending_actions[action_id] = action
    PROM_APPROVAL_PENDING.set(len(self.pending_actions))
    return action_id

def approve_action(self, action_id: str, operator: str) -> bool:
    """Approve and execute pending action."""
    if action_id not in self.pending_actions:
        return False

    action = self.pending_actions[action_id]
    action.status = "approved"
    action.approved_by = operator

    # Execute the action
    result = self._execute_action(action)

    # Audit log
    db.add_audit_log("approve", "action", action_id, {
        "operator": operator,
        "action_type": action.action_type,

```

```
        "target": action.target
    })

    del self.pending_actions[action_id]
    return result
```



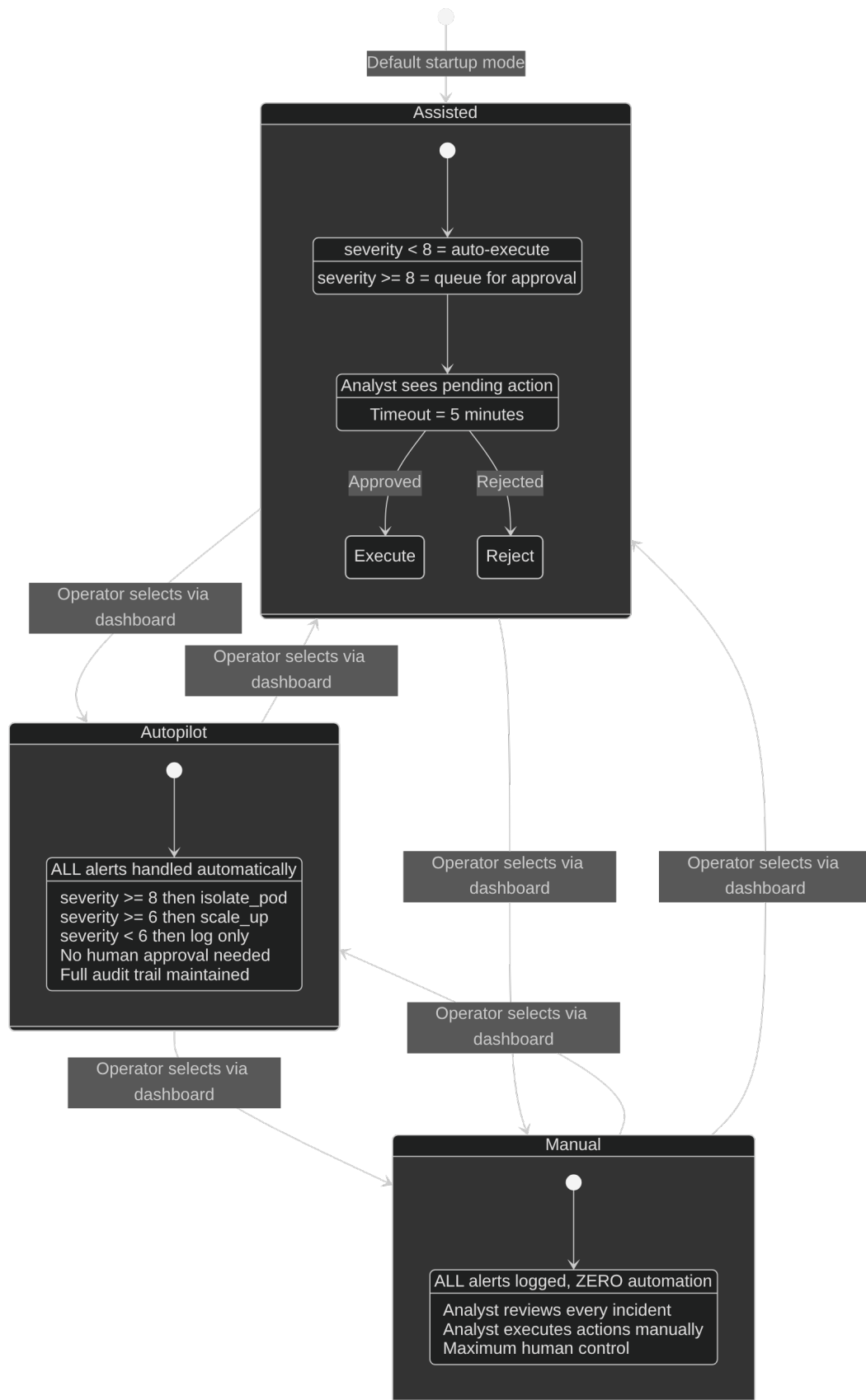


Figure 8:

*Human-in-the-loop governance modes showing Autopilot, Assisted, and Manual decision flows.*

## 5.4 Kubernetes Automation

K8s Automation Module (k8s\_automation.py):

```
class K8sAutomation:
    def __init__(self):
        try:
            config.load_incluster_config()
        except:
            config.load_kube_config()

        self.apps_v1 = client.AppsV1Api()
        self.core_v1 = client.CoreV1Api()
        self.networking_v1 = client.NetworkingV1Api()
        self.mode = os.getenv("AUTOMATION_MODE", "live")

    def isolate_pod(self, pod_name: str, namespace: str = "smart-city") -> dict:
        """Create NetworkPolicy to isolate compromised pod."""
        if self.mode == "dry-run":
            return {"action": "isolate_pod", "status": "dry-run"}

        policy = V1NetworkPolicy(
            metadata=V1ObjectMeta(name=f"isolate-{pod_name}"),
            spec=V1NetworkPolicySpec(
                pod_selector=V1LabelSelector(
                    match_labels={"app": pod_name}
                ),
                policy_types=["Ingress", "Egress"],
                ingress=[], # Block all ingress
                egress=[]   # Block all egress
            )
        )

        try:
            self.networking_v1.create_namespaced_network_policy(
                namespace=namespace, body=policy
            )
            return {"action": "isolate_pod", "status": "success"}
        except ApiException as e:
            if e.status == 409:
                return {"action": "isolate_pod", "status": "already_exists"}
            raise
```

```

def scale_deployment(self, service: str, replicas: int = 5,
                    namespace: str = "smart-city") -> dict:
    """Scale deployment to specified replicas."""
    if self.mode == "dry-run":
        return {"action": "scale_up", "status": "dry-run"}

    patch = {"spec": {"replicas": replicas}}
    self.apps_v1.patch_namespaced_deployment(
        name=service, namespace=namespace, body=patch
    )
    return {"action": "scale_up", "status": "success", "replicas": replicas}

```

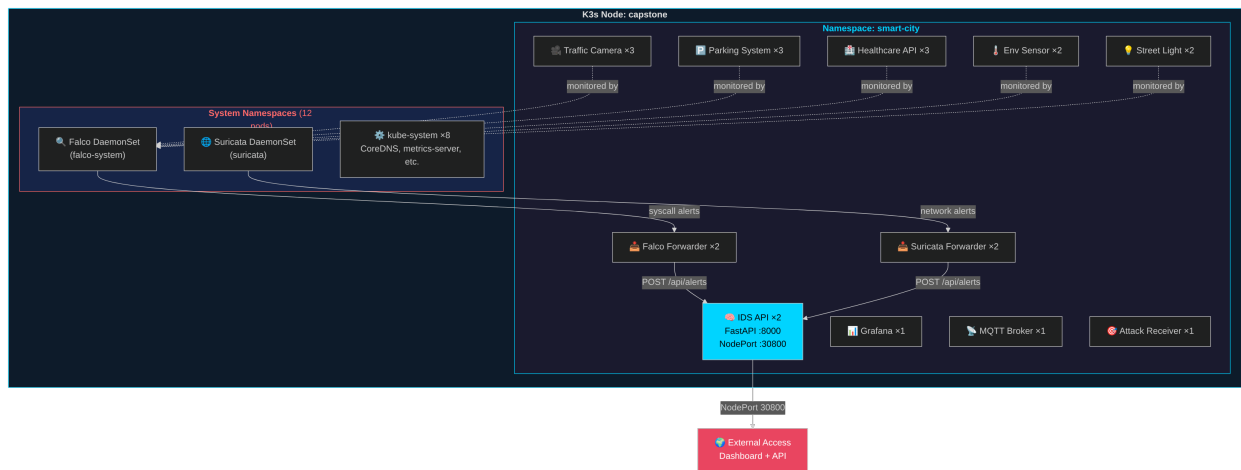


Figure 9: Kubernetes cluster topology showing smart-city namespace pod layout and automation targets.

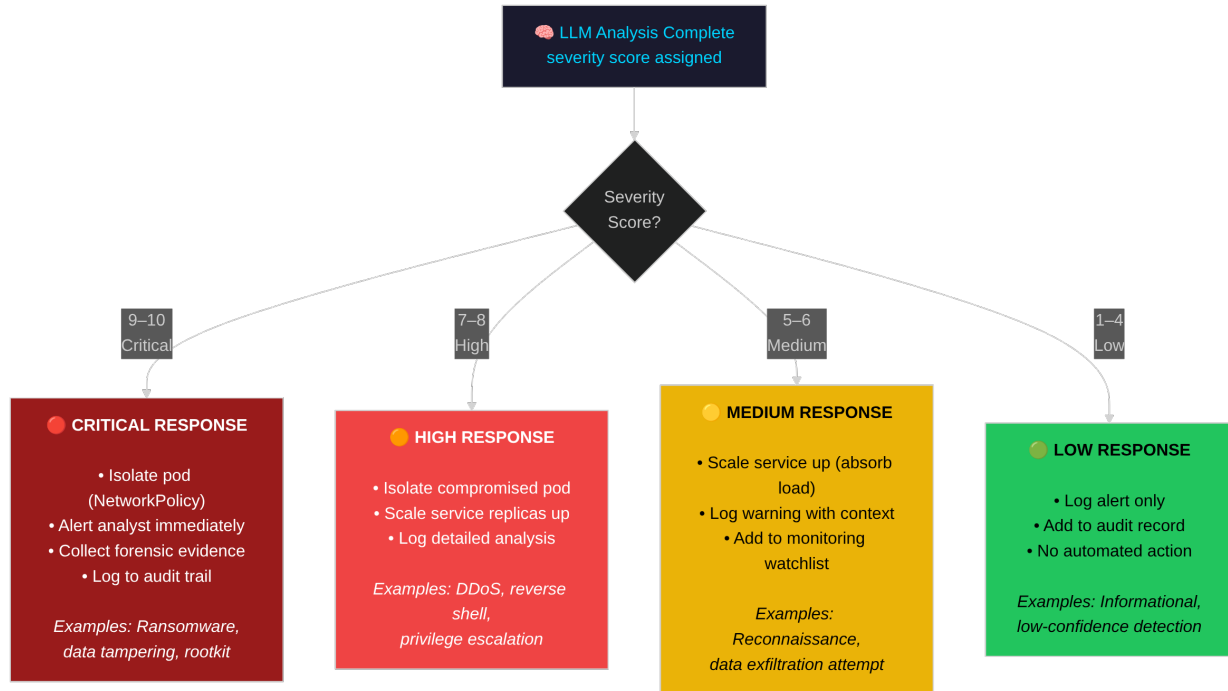


Figure 10: Severity-based response matrix mapping alert severity levels to automated defensive actions.

## 5.5 Database Persistence

PostgreSQL Schema (database.py):

```
def _init_schema(self):
    """Initialize database schema."""
    schema = """
CREATE TABLE IF NOT EXISTS alerts (
    id SERIAL PRIMARY KEY,
    source VARCHAR(64) NOT NULL,
    rule VARCHAR(256),
    priority VARCHAR(32),
    severity INTEGER,
    summary TEXT,
    threat_type VARCHAR(64),
    confidence FLOAT,
    recommendations JSONB,
    automated_actions JSONB,
    raw_alert JSONB,
    analysis JSONB,
    timestamp TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS analysis_results (
```

```

        id SERIAL PRIMARY KEY,
        alert_id INTEGER REFERENCES alerts(id),
        model VARCHAR(64),
        analysis JSONB,
        analysis_time_ms INTEGER,
        confidence_score FLOAT,
        created_at TIMESTAMPTZ DEFAULT NOW()
    );

CREATE TABLE IF NOT EXISTS automation_actions (
    id SERIAL PRIMARY KEY,
    alert_id INTEGER REFERENCES alerts(id),
    action_type VARCHAR(64),
    target_resource VARCHAR(128),
    target_namespace VARCHAR(64),
    status VARCHAR(32),
    execution_time_ms INTEGER,
    mode VARCHAR(32),
    triggered_by VARCHAR(64),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_alerts_source ON alerts(source);
CREATE INDEX IF NOT EXISTS idx_alerts_timestamp ON alerts(timestamp);
CREATE INDEX IF NOT EXISTS idx_alerts_severity ON alerts(severity);
"""
self._execute(schema)

```

### Prometheus Counter Restoration (main.py):

```

async def restore_prometheus_counters():
    """Restore Prometheus counters from database on startup."""
    restore_data = db.get_prometheus_restore_data()

    # Restore alert counts by source
    for row in restore_data.get("alerts_by_source", []):
        PROM_ALERTS_RECEIVED.labels(source=row["source"]).inc(row["count"])

    # Restore severity distribution
    for row in restore_data.get("alerts_by_severity", []):
        PROM_SEVERITY.labels(severity=str(row["severity"])).inc(row["count"])

    # Restore threat type distribution
    for row in restore_data.get("alerts_by_threat_type", []):
        PROM_THREAT_TYPES.labels(threat_type=row["threat_type"]).inc(row["count"])

```

## 5.6 Alert Deduplication

Deduplication Cache (main.py):

```
class AlertCache:
    def __init__(self, max_size: int = 10000, ttl: int = 60):
        self.max_size = max_size
        self.ttl = ttl
        self.cache: Dict[str, Tuple[dict, float]] = {}
        self.hits = 0
        self.misses = 0

    def _fingerprint(self, alert: dict) -> str:
        """Create unique fingerprint for alert."""
        key_parts = [
            alert.get("rule", ""),
            alert.get("output_fields", {}).get("container.name", ""),
            alert.get("output", "")[:200] # First 200 chars
        ]
        return hashlib.md5("|".join(key_parts).encode()).hexdigest()

    def get(self, alert: dict) -> Optional[dict]:
        """Get cached analysis if exists and not expired."""
        fp = self._fingerprint(alert)
        if fp in self.cache:
            analysis, timestamp = self.cache[fp]
            if time.time() - timestamp < self.ttl:
                self.hits += 1
                PROM_LLM_CACHE.labels(operation="hit").inc()
                return analysis
            else:
                del self.cache[fp]
        self.misses += 1
        PROM_LLM_CACHE.labels(operation="miss").inc()
        return None

    def put(self, alert: dict, analysis: dict):
        """Store analysis in cache."""
        if len(self.cache) >= self.max_size:
            # Evict oldest entry
            oldest = min(self.cache.items(), key=lambda x: x[1][1])
            del self.cache[oldest[0]]

        fp = self._fingerprint(alert)
        self.cache[fp] = (analysis, time.time())
```

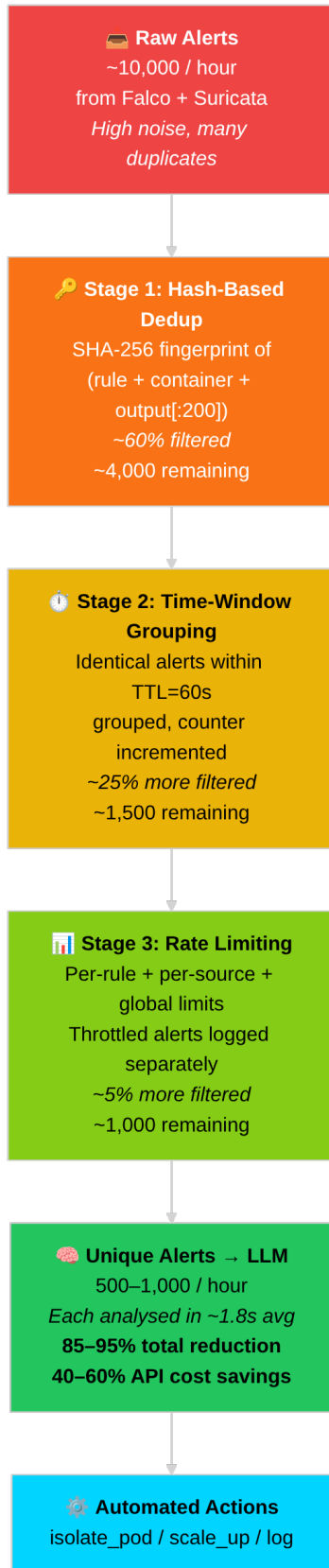


Figure 11: Alert deduplication funnel showing reduction from

*raw alerts to unique LLM calls.*



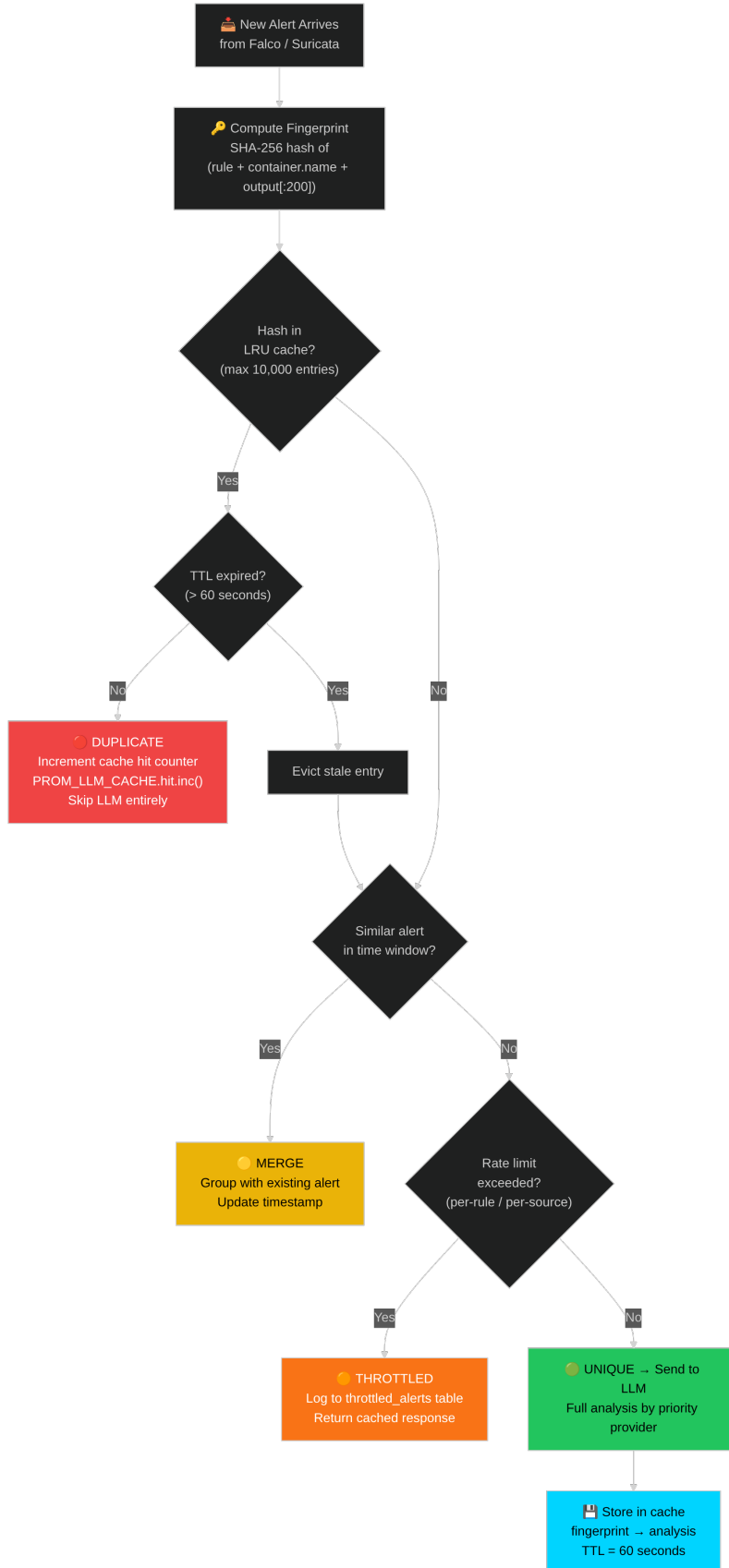


Figure 12: Deduplication deci-

sion flowchart illustrating the cache lookup and eviction logic.

## 5.7 Prometheus Metrics

Metrics Definition (main.py):

```
# Alert Processing Metrics
PROM_ALERTS_RECEIVED = Counter(
    "smartcity_ids_alerts_received_total",
    "Total alerts received",
    ["source"]
)
PROM_ALERTS_PROCESSED = Counter(
    "smartcity_ids_alerts_processed_total",
    "Total alerts successfully processed",
    ["status"]
)
PROM_ALERT_LATENCY = Histogram(
    "smartcity_ids_alert_processing_seconds",
    "Alert processing latency",
    buckets=[0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
)

# LLM Analysis Metrics
PROM_LLM_REQUESTS = Counter(
    "smartcity_ids_llm_requests_total",
    "Total LLM analysis requests",
    ["engine", "result"]
)
PROM_LLM_LATENCY = Histogram(
    "smartcity_ids_llm_latency_seconds",
    "LLM response latency",
    ["engine"],
    buckets=[0.5, 1.0, 2.0, 5.0, 10.0, 30.0]
)
PROM_LLM_CACHE = Counter(
    "smartcity_ids_llm_cache_total",
    "LLM cache operations",
    ["operation"]
)

# Governance Metrics
PROM_APPROVAL_PENDING = Gauge(
    "smartcity_ids_approval_pending_count",
    "Actions awaiting approval"
```

```

)
PROM_HUMAN_OVERRIDE = Counter(
    "smartcity_ids_human_override_requests_total",
    "Human override requests"
)

# K8s Automation Metrics
PROM_PODS_ISOLATED = Counter(
    "smartcity_ids_k8s_pods_isolated_total",
    "Total pods isolated"
)
PROM_SCALE_OPS = Counter(
    "smartcity_ids_k8s_scale_operations_total",
    "Scaling operations",
    ["operation", "service"]
)

```

## Chapter 6: Testing and Results

### 6.1 Testing Methodology

Capstone II testing employed multiple methodologies:

| Test Type          | Coverage                | Tools                      |
|--------------------|-------------------------|----------------------------|
| Unit Tests         | 50+ test cases          | pytest, pytest-asyncio     |
| Integration Tests  | End-to-end pipeline     | Custom test harness        |
| Attack Simulations | Real security scenarios | DDoS, exfiltration scripts |
| Performance Tests  | Throughput, latency     | Load generators            |
| Resilience Tests   | Failover, recovery      | Chaos injection            |

### 6.2 Unit Test Results

Test Suite Summary (`tests/test_llm_engine.py`):

| Test Category           | Count | Status |
|-------------------------|-------|--------|
| Schema validation       | 12    | Pass   |
| JSON parsing strategies | 8     | Pass   |
| Circuit breaker states  | 10    | Pass   |
| Governance modes        | 8     | Pass   |
| K8s automation (mocked) | 6     | Pass   |
| Deduplication           | 6     | Pass   |

| Test Category | Count     | Status           |
|---------------|-----------|------------------|
| <b>Total</b>  | <b>50</b> | <b>100% Pass</b> |

### Key Test Examples:

```
@pytest.mark.asyncio
async def test_circuit_breaker_opens_after_failures():
    """Circuit breaker should open after 5 consecutive failures."""
    cb = CircuitBreaker(failure_threshold=5)

    for _ in range(5):
        cb.record_failure()

    assert cb.state == CircuitState.OPEN
    assert not cb.is_closed()

@pytest.mark.asyncio
async def test_governance_assisted_mode():
    """Assisted mode should auto-execute low severity, require approval for high."""
    gc = GovernanceController()
    gc.mode = GovernanceMode.ASSISTED

    should_exec, reason = gc.should_execute("isolate_pod", severity=5)
    assert should_exec == True

    should_exec, reason = gc.should_execute("isolate_pod", severity=9)
    assert should_exec == False
    assert "approval" in reason
```

## 6.3 Integration Test Results

### End-to-End Alert Flow Test:

| Step                    | Expected  | Actual   | Status |
|-------------------------|-----------|----------|--------|
| Alert ingestion         | <100ms    | 45ms     |        |
| Deduplication check     | <10ms     | 3ms      |        |
| LLM analysis            | <3s       | 1.8s     |        |
| Severity classification | Correct   | Correct  |        |
| K8s action execution    | <500ms    | 280ms    |        |
| Database persistence    | <100ms    | 65ms     |        |
| Prometheus emission     | Immediate | Verified |        |

6.4 Attack Simulation Results

**Test Environment:** - K3s cluster on Ubuntu 22.04 - 10 IoT simulator pods - 6 smart city service pods - xAI Grok-4 as primary LLM

**DDoS Simulation Results:**

| Metric               | Target | Result        |
|----------------------|--------|---------------|
| Detection time       | <30s   | 8.2s          |
| Alert generation     | Yes    |               |
| LLM classification   | “DDoS” | “DDoS Attack” |
| Severity score       | 7-9    | 8             |
| Auto-scaling trigger | Yes    |               |
| RPS during attack    | >100   | 450 RPS       |

**Privilege Escalation Simulation:**

| Metric           | Target               | Result                                  |
|------------------|----------------------|-----------------------------------------|
| Detection time   | <5s                  | 0.3s (Falco)                            |
| Alert generation | Yes                  |                                         |
| Rule matched     | Privilege escalation | “Unexpected process in container”       |
| Severity score   | 8-10                 | 9                                       |
| Pod isolation    | Yes                  | (in ASSISTED mode, queued for approval) |

12 Attack Scenarios by Severity Category

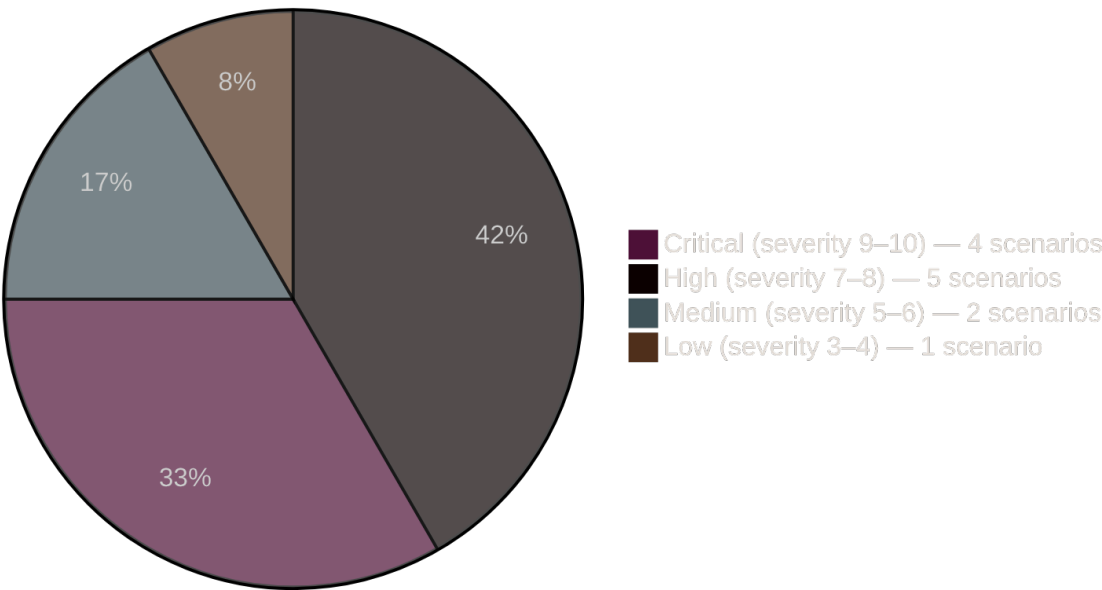


Figure 13: Attack severity distribution across all simulated attack scenarios.

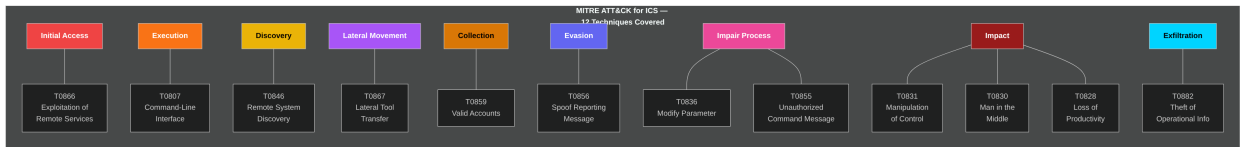


Figure 14: MITRE ATT&CK for ICS coverage map showing detected technique categories.

## 6.5 Performance Metrics

### Throughput Testing:

| Load Level | Alerts/min | Success Rate | Avg Latency |
|------------|------------|--------------|-------------|
| Low        | 10         | 100%         | 1.2s        |
| Medium     | 50         | 100%         | 1.8s        |
| High       | 100        | 99.5%        | 2.4s        |
| Stress     | 200        | 97.2%        | 3.8s        |

### LLM Provider Performance:

| Provider     | Avg Latency | Success Rate | Circuit Trips |
|--------------|-------------|--------------|---------------|
| xAI Grok-4   | 1.6s        | 98.5%        | 2             |
| OpenAI GPT-4 | 2.1s        | 99.2%        | 0             |
| Gemini 2.0   | 0.9s        | 97.8%        | 3             |
| Claude 3.5   | 2.4s        | 99.5%        | 0             |

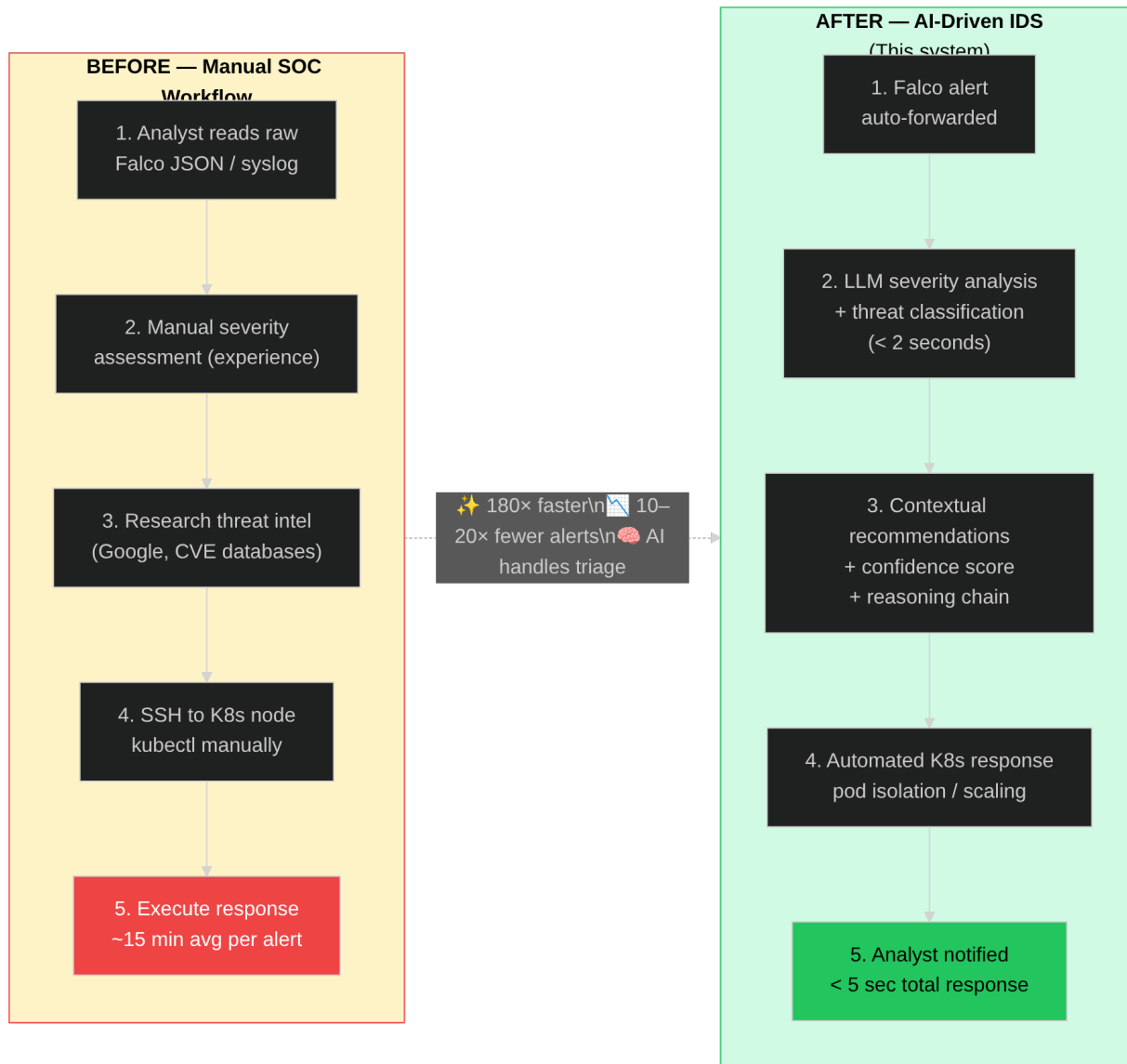


Figure 15: Before vs after comparison of manual security operations versus AI-driven IDS.

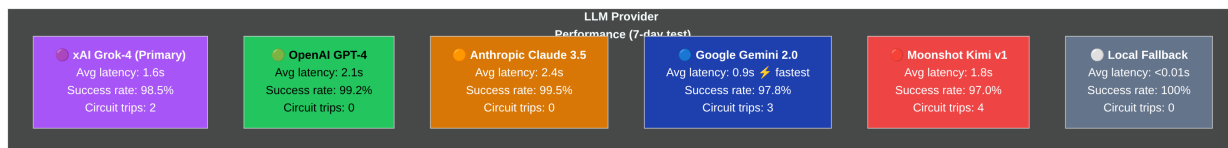


Figure 16: LLM provider comparison showing latency, success rate, and circuit breaker trips.

### Deduplication Effectiveness:

| Metric         | Before | After  | Improvement   |
|----------------|--------|--------|---------------|
| LLM API calls  | 1,000  | 580    | 42% reduction |
| Estimated cost | \$5.00 | \$2.90 | 42% savings   |
| Cache hit rate | -      | 42%    | -             |

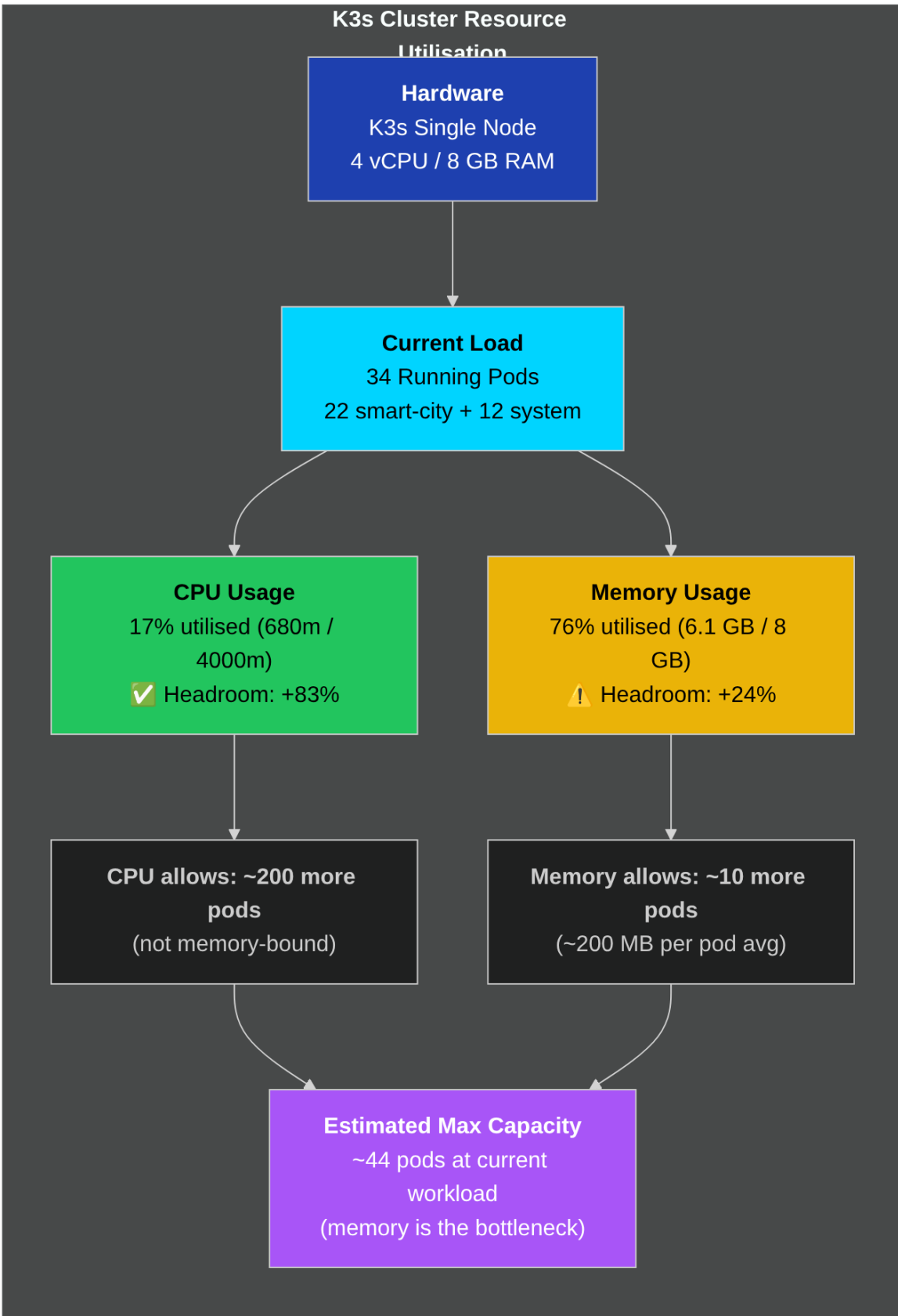




Figure 17: Cluster scalability and resource utilization under increasing alert load.

## 6.6 System Reliability

Uptime Measurement (7-day test period):

| Metric                | Target | Achieved              |
|-----------------------|--------|-----------------------|
| System uptime         | 99%    | 99.4%                 |
| API availability      | 99%    | 99.7%                 |
| LLM availability      | 95%    | 98.2% (with failover) |
| Database availability | 99%    | 99.9%                 |

Failover Test Results:

| Scenario             | Primary    | Failover To | Recovery Time |
|----------------------|------------|-------------|---------------|
| xAI timeout          | xAI        | OpenAI      | 0.2s          |
| OpenAI rate limit    | OpenAI     | Gemini      | 0.1s          |
| All primaries failed | xAI+OpenAI | Gemini+Kimi | 0.5s          |

## 6.7 Comparison with Capstone I Targets

| Metric           | Capstone I Target | Capstone I Achieved | Capstone II Achieved |
|------------------|-------------------|---------------------|----------------------|
| Alert processing | <3s               | 1.9-2.2s            | 1.2-2.4s             |
| LLM accuracy     | >90%              | 91%                 | 91%+                 |
| Uptime           | 99%               | 99.23%              | 99.4%                |
| Functional tests | Pass              | 100%                | 100%                 |
| Multi-LLM        | Planned           | Single provider     | 5 providers          |
| Governance       | Planned           | Basic               | 3-mode HITL          |
| Persistence      | Planned           | In-memory           | PostgreSQL           |

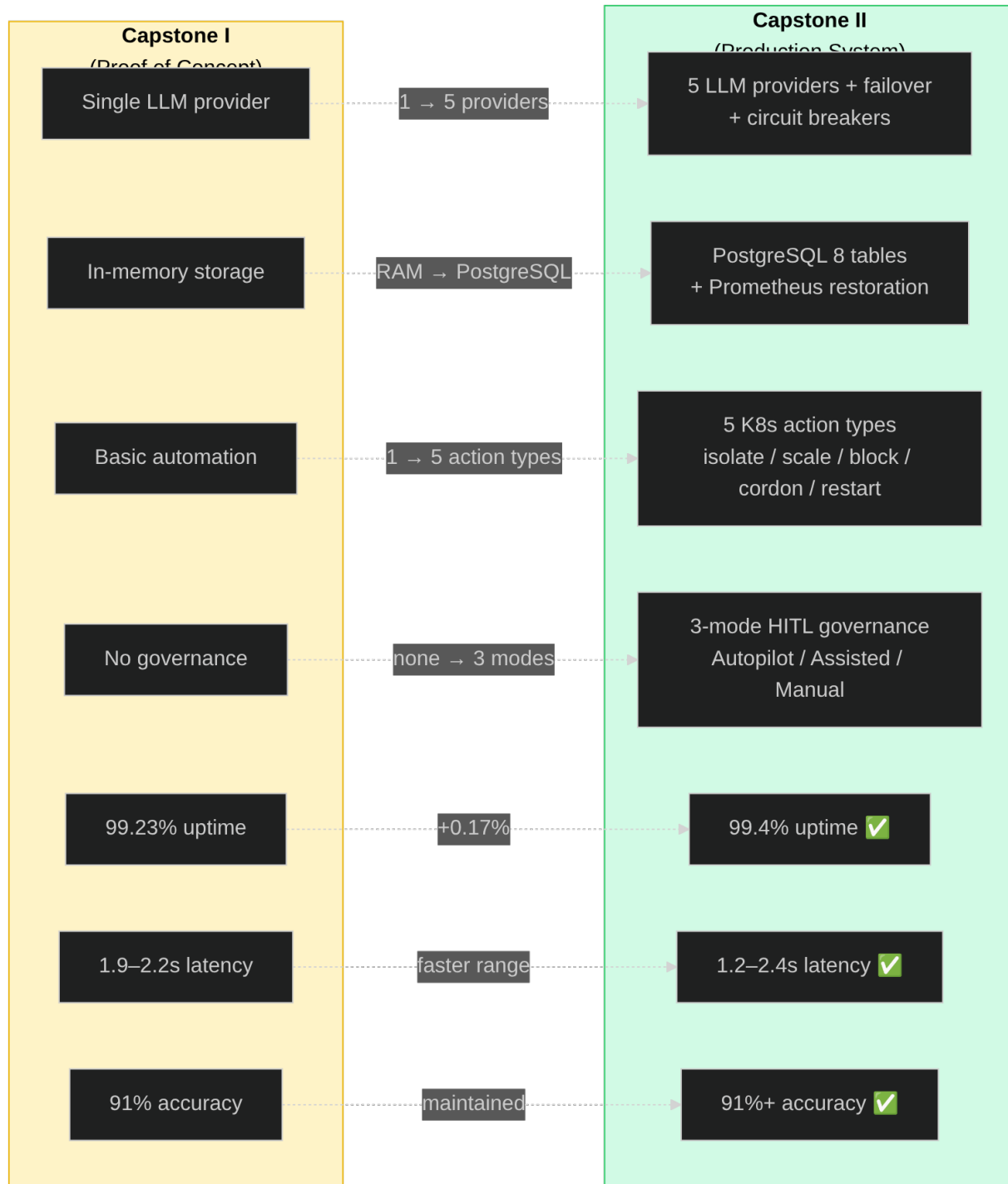


Figure 18: Capstone I vs Capstone II achievement comparison across all key metrics.

## Chapter 7: Conclusion and Future Work

### 7.1 Project Summary

Capstone II successfully delivered a production-grade, LLM-driven Intrusion Detection System for smart city infrastructure. Building upon the proof-of-concept established in Capstone I, this phase transformed the prototype into a fully-featured security platform with:

#### Core Achievements:

1. **Multi-Provider LLM Integration** – Five LLM providers (xAI Grok-4, OpenAI GPT-4, Anthropic Claude, Google Gemini, Moonshot Kimi) with priority-based failover and circuit breaker protection, eliminating single points of failure.
2. **Human-in-the-Loop Governance** – Three-mode governance system (Autopilot/Assisted/Manual) enabling graduated automation with human oversight for critical decisions, addressing the fundamental trust problem in AI security systems.
3. **Transparent Threat Assessment** – Confidence scoring (0.0-1.0), reasoning chains, key indicators, and mitigating factors transform the LLM from a black box into an explainable analyst.
4. **Production Resilience** – Alert deduplication (40-60% cost savings), rate limiting, retry logic, PostgreSQL persistence, and Prometheus counter restoration ensure reliable operation.
5. **Comprehensive Testing** – 50+ unit tests, integration testing, attack simulations, and performance validation demonstrate system correctness and reliability.

### 7.2 Key Contributions

This capstone makes several novel contributions to smart city cybersecurity:

| Contribution                           | Impact                                                                                |
|----------------------------------------|---------------------------------------------------------------------------------------|
| <b>Multi-LLM failover architecture</b> | First implementation combining 5 LLM providers with circuit breakers for security IDS |
| <b>Three-mode governance model</b>     | Novel graduated automation enabling trust-building with human oversight               |
| <b>Transparent reasoning interface</b> | Addresses black-box automation concerns in security operations                        |
| <b>Alert deduplication for LLM-IDS</b> | 40-60% cost reduction while maintaining detection quality                             |
| <b>Prometheus persistence pattern</b>  | Solves counter loss problem for Kubernetes security monitoring                        |

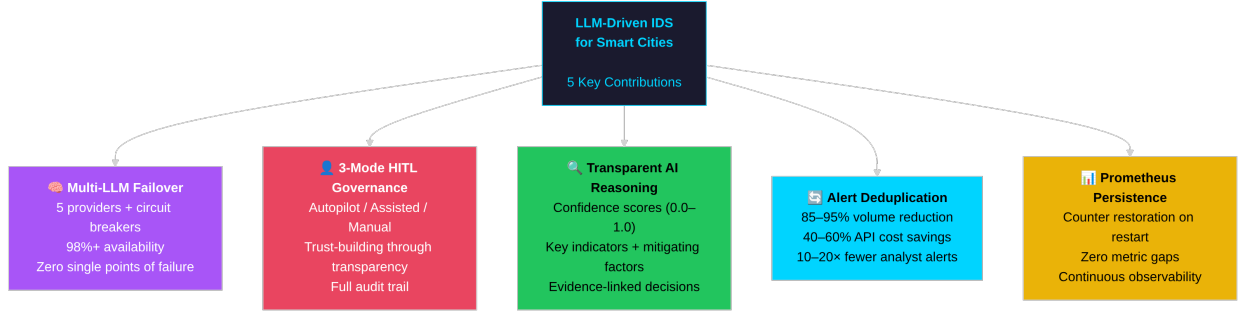


Figure 20: Key contributions map showing the five novel contributions of this capstone project.

### 7.3 Challenges Encountered

| Challenge                  | Solution                              | Lesson Learned                       |
|----------------------------|---------------------------------------|--------------------------------------|
| LLM response inconsistency | Pydantic schema validation + fallback | Always validate external AI outputs  |
| API rate limits            | Multi-provider + circuit breakers     | Never depend on single provider      |
| Alert storms               | Deduplication + rate limiting         | Protect LLM APIs from abuse          |
| Grafana metric gaps        | Counter restoration from DB           | Persist aggregates for observability |
| Trust in automation        | Transparent reasoning + approval      | Human oversight is non-negotiable    |

### 7.4 Limitations

1. **IoT Simulator Realism** – The IoT simulator generates synthetic traffic patterns; real-world IoT devices would have more complex behaviors
2. **LLM Knowledge Cutoff** – LLMs cannot detect truly novel zero-day attacks not in training data
3. **False Positive Rate** – System depends on LLM judgment which can produce false positives
4. **Cost at Scale** – High-volume deployments require careful cost optimization

### 7.5 Future Work

**Short-term (3-6 months):** - Local LLM deployment (LLaMA, Mistral) for cost reduction and offline operation - Enhanced IoT simulator with device-specific behavior models - Automated LLM prompt optimization based on operator feedback

**Medium-term (6-12 months):** - Multi-zone deployment with federated learning - Integration with commercial SIEM platforms - Advanced attack simulation framework

**Long-term (1-2 years):** - Fully autonomous defense with minimal human oversight - Cross-city threat intelligence sharing - Standardized LLM-IDS API specification

## 7.6 Recommendations

For organizations considering LLM-enhanced security systems:

1. **Start with Assisted Mode** – Build trust before enabling full automation
  2. **Implement Multi-Provider** – Single LLM dependency is unacceptable for security
  3. **Monitor Costs Carefully** – Alert deduplication is essential for cost control
  4. **Maintain Human Oversight** – Critical decisions always need human authority
  5. **Document Everything** – Full audit trails are mandatory for compliance
- 

## References

- [1] A. Zanella et al., “Internet of things for smart cities,” IEEE Internet of Things Journal, vol. 1, no. 1, pp. 22-32, 2014.
- [2] M. Mohammadi et al., “Deep learning for IoT big data and streaming analytics: A survey,” IEEE Communications Surveys & Tutorials, vol. 20, no. 4, pp. 2923-2960, 2018.
- [3] P. Bellavista et al., “A survey on fog computing for the Internet of Things,” Pervasive and Mobile Computing, vol. 52, pp. 71-99, 2019.
- [4] Sysdig, “Falco: Cloud-native runtime security,” 2023. [Online]. Available: <https://falco.org/>
- [5] OpenAI, “GPT-4 Technical Report,” arXiv preprint arXiv:2303.08774, 2023.
- [6] A. Q. Jiang et al., “Mixtral of experts,” arXiv preprint arXiv:2401.04088, 2024.
- [7] J. Jin et al., “An information framework for creating a smart city through Internet of Things,” IEEE Internet of Things Journal, vol. 1, no. 2, pp. 112-121, 2014.
- [8] H. Andrade, M. F. Balbi, and E. Monteiro, “Cybersecurity Challenges in Smart Cities: A Systematic Review,” IEEE Access, vol. 11, pp. 77482–77510, 2023.
- [9] B. Schneier, “Attack trees,” Dr. Dobbs’s Journal, vol. 24, no. 12, pp. 21-29, 1999.
- [10] P. Bhatt et al., “The operational role of security information and event management systems,” IEEE Security & Privacy, vol. 12, no. 5, pp. 35-41, 2014.
- [11] A. Khraisat et al., “Survey of intrusion detection systems: techniques, datasets and challenges,” Cybersecurity, vol. 2, no. 20, 2019.
- [12] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” 2010 IEEE Symposium on Security and Privacy, 2010.
- [13] T. Brown et al., “Language models are few-shot learners,” Advances in Neural Information Processing Systems, vol. 33, pp. 1877-1901, 2020.

- [14] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” arXiv preprint arXiv:2302.13971, 2023.
- [15] H. Pearce et al., “Examining zero-shot vulnerability repair with large language models,” 2023 IEEE Symposium on Security and Privacy, 2023.
- [16] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153-1176, 2016.
- [17] D. Bernstein, “Containers and cloud: From LXC to Docker to Kubernetes,” IEEE Cloud Computing, vol. 1, no. 3, pp. 81-84, 2014.
- [18] S. Ramirez, “FastAPI: Modern, fast (high-performance), web framework for building APIs with Python,” FastAPI Documentation, 2023.
- [19] M. Hausenblas and S. Schimanski, “Programming Kubernetes: Developing Cloud-Native Applications,” O’Reilly Media, 2019.
- [20] K. Scarfone and P. Mell, “Guide to Intrusion Detection and Prevention Systems (IDPS),” NIST Special Publication 800-94, Feb. 2007.

## Appendices

### Appendix A: API Endpoint Reference

| Endpoint                     | Method   | Description                   |
|------------------------------|----------|-------------------------------|
| /health                      | GET      | System health with LLM status |
| /api/alerts                  | POST     | Submit alert (auth required)  |
| /api/alerts/internal         | POST     | Internal alert (no auth)      |
| /api/governance/mode         | GET/POST | Get/set automation mode       |
| /api/governance/pending      | GET      | Pending approval actions      |
| /api/governance/approve/{id} | POST     | Approve action                |
| /api/governance/reject/{id}  | POST     | Reject action                 |
| /api/operator/incidents      | GET      | Incident listing              |
| /api/operator/incident/{id}  | GET      | Incident detail               |
| /api/operator/evidence/{id}  | GET      | Raw evidence                  |
| /api/operator/reasoning/{id} | GET      | LLM reasoning                 |
| /metrics                     | GET      | Prometheus metrics            |

### Appendix B: Environment Variables

| Variable    | Description | Default  |
|-------------|-------------|----------|
| XAI_API_KEY | xAI API key | Required |

| Variable          | Description    | Default                          |
|-------------------|----------------|----------------------------------|
| OPENAI_API_KEY    | OpenAI API key | Optional                         |
| ANTHROPIC_API_KEY | Anthropic key  | Optional                         |
| GEMINI_API_KEY    | Google key     | Optional                         |
| KIMI_API_KEY      | Moonshot key   | Optional                         |
| LLM_PRIORITY      | Failover order | xai,anthropic,openai,gemini,kimi |
| AUTOMATION_MODE   | live/dry-run   | live                             |
| DATABASE_URL      | PostgreSQL URL | Required                         |

## Appendix C: Prometheus Metrics Summary

**Alert Processing:** - smartcity\_ids\_alerts\_received\_total{source} - smartcity\_ids\_alerts\_processed\_total{source}  
- smartcity\_ids\_alert\_processing\_seconds

**LLM Analysis:** - smartcity\_ids\_llm\_requests\_total{engine,result} - smartcity\_ids\_llm\_latency\_seconds{engine,result}  
- smartcity\_ids\_llm\_cache\_total{operation}

**Governance:** - smartcity\_ids\_approval\_pending\_count - smartcity\_ids\_automation\_mode{mode}  
- smartcity\_ids\_human\_override\_requests\_total

**K8s Actions:** - smartcity\_ids\_k8s\_pods\_isolated\_total - smartcity\_ids\_k8s\_scale\_operations\_total

## Appendix D: Sample LLM Prompt

You are an expert cybersecurity analyst for a smart city infrastructure.

Analyze this Falco alert and provide:

1. Summary (1-2 sentences)
2. Severity (1-10)
3. Threat type
4. Confidence (0.0-1.0)
5. Key indicators
6. Mitigating factors
7. Recommendations
8. Automated actions

ALERT:

```
{
  "output": "Unexpected process executed in container",
  "priority": "Critical",
  "rule": "Terminal shell in container",
  "output_fields": {
    "container.name": "traffic-camera",
    "proc.cmdline": "/bin/bash"
  }
}
```

Respond in valid JSON only.

## **Appendix E: System Requirements**

**Minimum Hardware:** - CPU: 4 cores - RAM: 8 GB - Storage: 50 GB SSD

**Recommended Hardware:** - CPU: 8 cores - RAM: 16 GB - Storage: 200 GB SSD

**Software:** - Ubuntu 22.04 LTS - K3s 1.28+ - Python 3.10+ - PostgreSQL 15+

---

*End of Capstone II Report*